

Number Systems



Number System

- The technique to represent and work with numbers is called **number system**.
 - **Decimal number system** is the most common number system.
 - Other popular number systems include **binary number system, octal number system, hexadecimal number system**, etc.
 - Computers cannot understand human languages, so **to understand the commands and instructions given to the computers by programmers**, different number systems are used such as binary, octal, decimal, etc.
 - **Types of Number Systems**
 - There are different types of number systems in which the four main types are as follows.
 1. Binary number system (Base - 2)
 2. Octal number system (Base - 8)
 3. Decimal number system (Base - 10)
 4. Hexadecimal number system (Base - 16)
-

Number System

Key Components of a Number System:

Every number system is based on:

1. **Base (or Radix)** – total number of unique digits it uses.
2. **Digits / Symbols** – characters used to represent numbers.
3. **Place value** – the position of each digit determines its value.

Types of Number Systems (Used in Computers):

Number System	Base (Radix)	Digits / Symbols Used	Example	Used In
Decimal	10	0–9	245, 19.5	Humans (daily life)
Binary	2	0, 1	1011 ₂	Computers (machine language)
Octal	8	0–7	745 ₈	Short form of binary
Hexadecimal	16	0–9, A–F	2A3 ₁₆	Memory addresses, color codes

Number System

Decimal number 25 in other systems:

System	Representation	Explanation
Decimal	25	$(2 \times 10^1 + 5 \times 10^0)$
Binary	11001 ₂	$(1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0)$
Octal	31 ₈	$(3 \times 8^1 + 1 \times 8^0)$
Hexadecimal	19 ₁₆	$(1 \times 16^1 + 9 \times 16^0)$

Why Number Systems Matter in Computers:

- Computers **understand only binary** (0 and 1).
 - Other systems (octal, hexadecimal) make it easier for humans to read or write binary data.
 - Used in **memory addressing, programming, networking, and data representation**.
-

Number System

👉 Computers understand only the Binary Number System (0 and 1).

But humans use Octal (base 8) and Hexadecimal (base 16) systems for convenience — to make binary numbers shorter, readable, and easier to work with.

🧠 1. Binary is very long and hard to read

Binary numbers use only 0s and 1s, so even small values need many digits.

Example:

Decimal	Binary
255	11111111
4095	111111111111

That's difficult for humans to read or debug in programs or hardware addresses.

Number System

2. Octal and Hexadecimal make binary shorter

- Octal groups 3 binary bits = 1 octal digit
- Hexadecimal groups 4 binary bits = 1 hexadecimal digit

Example:

Binary: 1111 1111

- In Octal → 377
- In Hexadecimal → FF

System	Where It's Used	Example
Hexadecimal	Memory addresses, color codes, machine instructions	0x1A3F , #FF0000
Octal	File permissions in Unix/Linux	chmod 755
Binary	Actual data processing by CPU	10101010

Quantities/Counting (1 of 3)

Decimal	Binary	Octal	Hexa- decimal
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7

Quantities/Counting (2 of 3)

Decimal	Binary	Octal	Hexa- decimal
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Quantities/Counting (3 of 3)

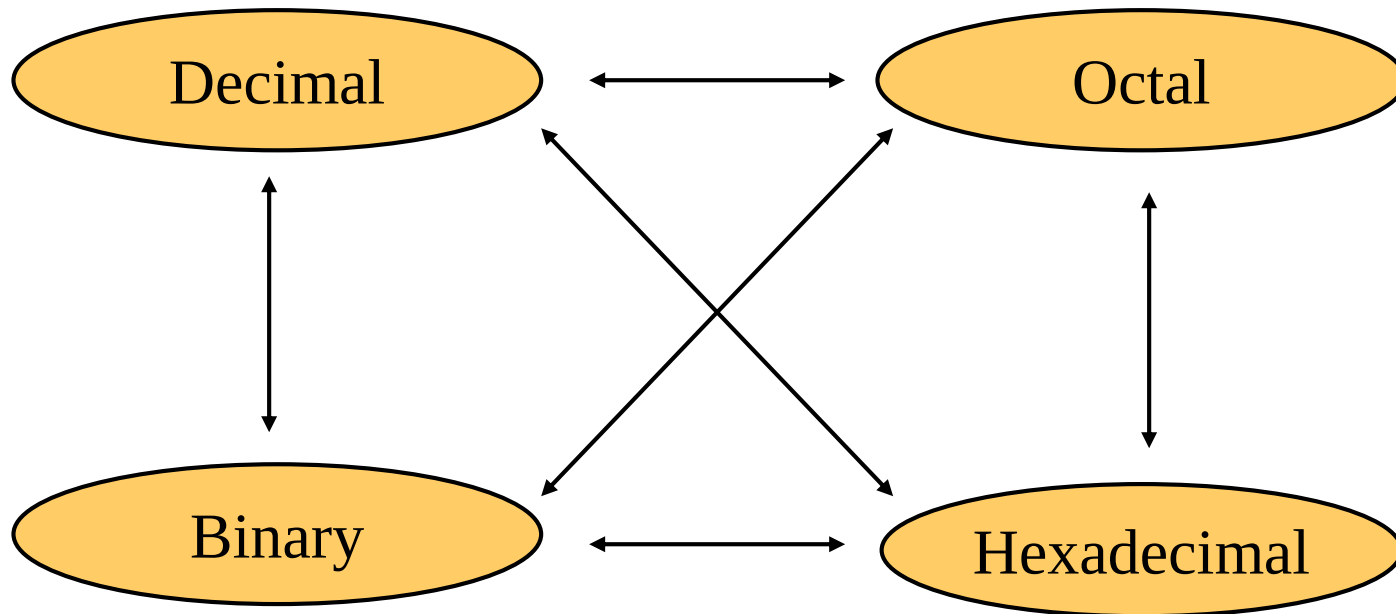
Decimal	Binary	Octal	Hexa- decimal
16	10000	20	10
17	10001	21	11
18	10010	22	12
19	10011	23	13
20	10100	24	14
21	10101	25	15
22	10110	26	16
23	10111	27	17

Etc.



Conversion Among Bases

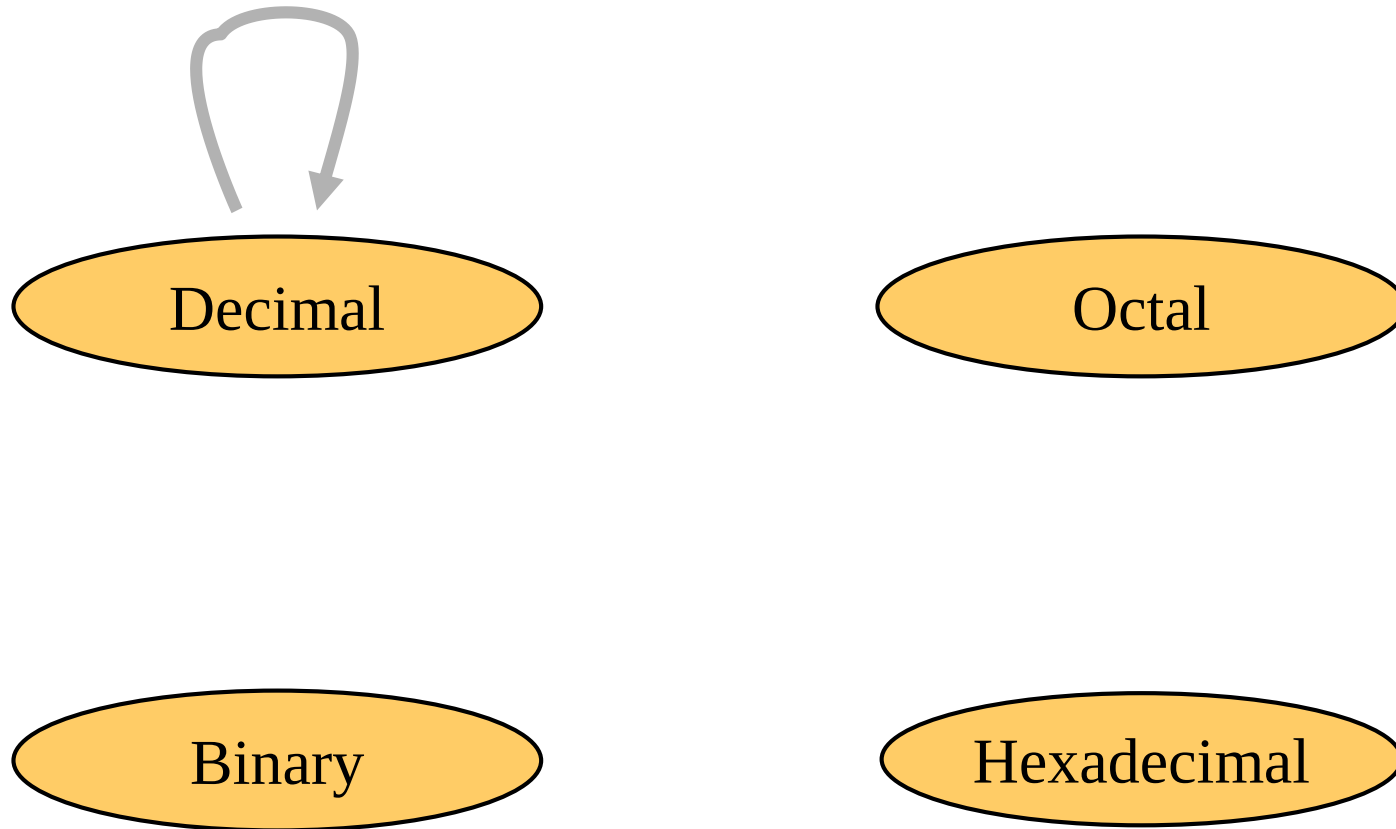
- The possibilities:



Conversion Among Bases

From → To	Method
Decimal → Any	Divide by base, take remainders
Any → Decimal	Multiply each digit by base ⁿ
Binary ↔ Octal	Group 3 bits
Binary ↔ Hexadecimal	Group 4 bits
Octal ↔ Hexadecimal	Use Binary as intermediate
Fractions	Multiply by base, take integer parts

Decimal to Decimal



Next slide...

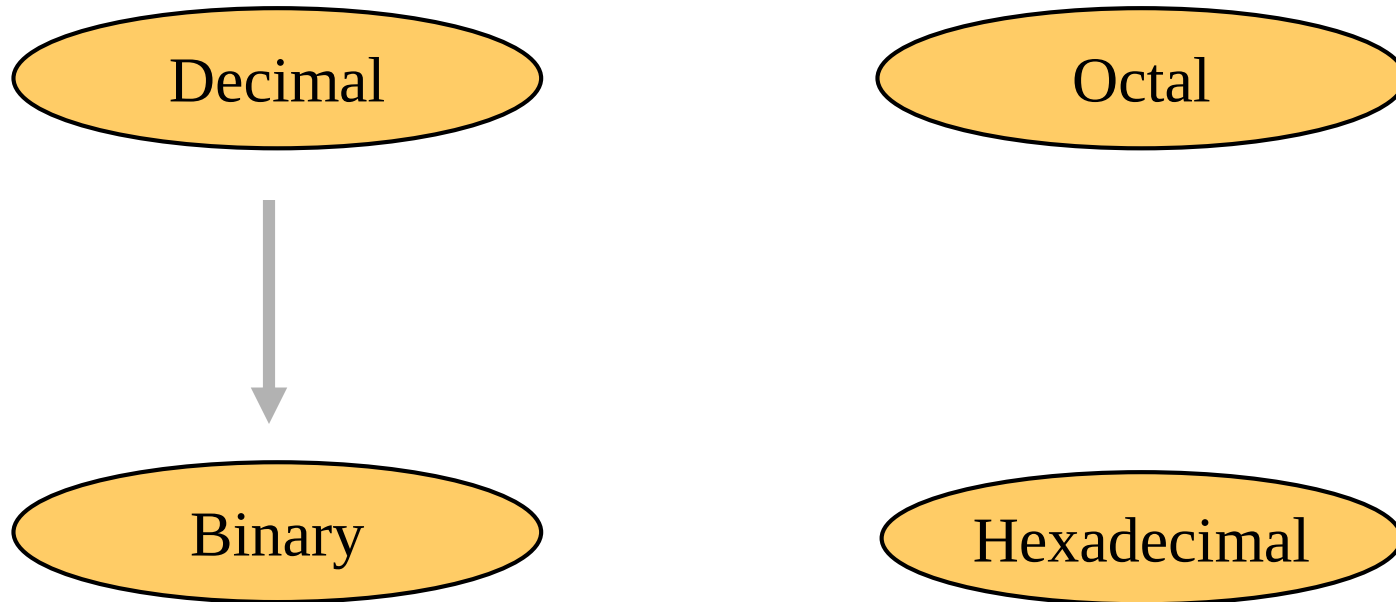
$125_{10} \Rightarrow$

5	$\times$	10^0	=	5
2	$\times$	10^1	=	20
1	$\times$	10^2	=	100
				125

Weight

Base

Decimal to Binary



Decimal to Binary

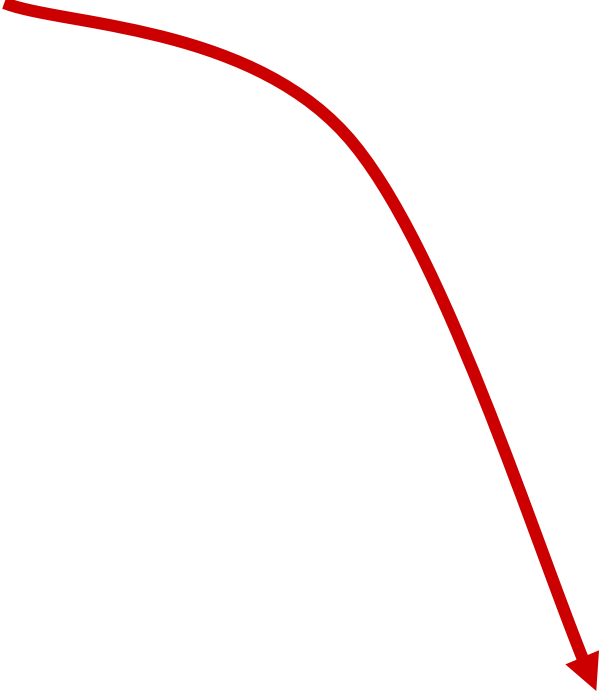
- Technique
 - Divide by two, keep track of the remainder



Example

$$125_{10} = ?_2$$

2		125	
2		62	1
2		31	0
2		15	1
2		7	1
2		3	1
2		1	1
		0	1


$$125_{10} = 1111101_2$$

Convert 56_{10} into a binary number.

$$56_{10} = ?_2$$

Division by 2	Quotient	Remainder	Binary Bit
$56 \div 2$	28	0	0(LSB)
$28 \div 2$	14	0	0
$14 \div 2$	7	0	0
$7 \div 2$	3	1	1
$3 \div 2$	1	1	1
$1 \div 2$	0	1	1

$$\therefore (56)_{10} = (111000)_2$$

Convert 278_{10} into a binary number.

$$278_{10} = ?_2$$

Division by 2	Quotient	Remainder	Binary Bit
$278 \div 2$	139	0	0 (LSB)
$139 \div 2$	69	1	1
$69 \div 2$	34	1	1
$34 \div 2$	17	0	0
$17 \div 2$	8	1	1
$8 \div 2$	4	0	0
$4 \div 2$	2	0	0
$2 \div 2$	1	0	0
$1 \div 2$	0	1	1 (MSB)

Solution:

$$\therefore (278)_{10} = (100010110)_2$$

Convert 180_{10} into a binary number.

$$180_{10} = ?_2$$

Division by 2	Quotient	Remainder	Binary Bit
$180 \div 2$	90	0	0(LSB)
$90 \div 2$	45	0	0
$45 \div 2$	22	1	1
$22 \div 2$	11	0	0
$11 \div 2$	5	1	1
$5 \div 2$	2	1	1
$2 \div 2$	1	0	0
$1 \div 2$	0	1	1(MSB)

$$\therefore (180)_{10} = (10110100)_2$$

Convert 1073_{10} into a binary number.

$$1073_{10} = ?_2$$

Division by 2	Quotient	Remainder	Binary Bit
$1073 \div 2$	536	1	1 (LSB)
$536 \div 2$	268	0	0
$268 \div 2$	134	0	0
$134 \div 2$	67	0	0
$67 \div 2$	33	1	1
$33 \div 2$	16	1	1
$16 \div 2$	8	0	0
$8 \div 2$	4	0	0
$4 \div 2$	2	0	0
$2 \div 2$	1	0	0
$1 \div 2$	0	1	1

$$\therefore (1073)_{10} = (10000110001)_2$$



Convert 81_{10} into a binary number.

$$81_{10} = ?_2$$

Division by 2	Quotient	Remainder	Binary Bit
$81 \div 2$	40	1	1(LSB)
$40 \div 2$	20	0	0
$20 \div 2$	10	0	0
$10 \div 2$	5	0	0
$5 \div 2$	2	1	1
$2 \div 2$	1	0	0
$1 \div 2$	0	1	1(MSB)

$$\therefore (81)_{10} = (1010001)_2$$

Convert 403_{10} into a binary number.

$$403_{10} = ?_2$$

Division by 2	Quotient	Remainder	Binary Bit
$403 \div 2$	201	1	1(LSB)
$201 \div 2$	100	1	1
$100 \div 2$	50	0	0
$50 \div 2$	25	0	0
$25 \div 2$	12	1	1
$12 \div 2$	6	0	0
$6 \div 2$	3	0	0
$3 \div 2$	1	1	1
$1 \div 2$	0	1	1(MSB)

$$\therefore (403)_{10} = (110010011)_2$$



Convert 508_{10} into a binary number.

$$508_{10} = ?_2$$

Division by 2	Quotient	Remainder	Binary Bit
$508 \div 2$	254	0	0(LSB)
$254 \div 2$	127	0	0
$127 \div 2$	63	1	1
$63 \div 2$	31	1	1
$31 \div 2$	15	1	1
$15 \div 2$	7	1	1
$7 \div 2$	3	1	1
$3 \div 2$	1	1	1
$1 \div 2$	0	1	1(MSB)

$$\therefore (508)_{10} = (111111100)_2$$



Convert 1278_{10} into a binary number.

$$1278_{10} = ?_2$$

Division by 2	Quotient	Remainder	Binary Bit
$1278 \div 2$	639	0	0(LSB)
$639 \div 2$	319	1	1
$319 \div 2$	159	1	1
$159 \div 2$	79	1	1
$79 \div 2$	39	1	1
$39 \div 2$	19	1	1
$19 \div 2$	9	1	1
$9 \div 2$	4	1	1
$4 \div 2$	2	0	0
$2 \div 2$	1	0	0
$1 \div 2$	0	1	1(MSB)

$$\therefore (1278)_{10} = (10011111110)_2$$



Convert 145_{10} into a binary number.

$$145_{10} = ?_2$$

Division by 2	Quotient	Remainder	Binary Bit
$145 \div 2$	72	1	1 (LSB)
$72 \div 2$	36	0	0
$36 \div 2$	18	0	0
$18 \div 2$	9	0	0
$9 \div 2$	4	1	1
$4 \div 2$	2	0	0
$2 \div 2$	1	0	0
$1 \div 2$	0	1	1 (MSB)

$$\therefore (145)_{10} = (10010001)_2$$



Practice Questions

Practice Questions on Decimal to Binary Conversions

1. Convert 155_{10} into a binary number.
 2. Convert 375_{10} into a binary number.
 3. Convert 74_{10} into a binary number.
-

Decimal(with fraction) to Binary

Example : 4.47

Step 1: Conversion of 4 to binary

1. $4/2$: Remainder = 0 : Quotient = 2
2. $2/2$: Remainder = 0 : Quotient = 1
3. $1/2$: Remainder = 1 : Quotient = 0

So equivalent binary of integral part of decimal is 100.

Step 2: Conversion of .47 to binary

1. $0.47 * 2 = 0.94$, Integral part: 0
2. $0.94 * 2 = 1.88$, Integral part: 1
3. $0.88 * 2 = 1.76$, Integral part: 1

So equivalent binary of fractional part of decimal is .011 (Repeat steps until you either get to 0 or a p

Step 3: Combined the result of step 1 and 2.

Final answer can be written as: 100.011

1 Fraction becomes 0

- The remaining fractional part is exactly 0 → conversion is **finished**.
- Example: 0.625

arduino

$0.625 \times 2 = 1.25 \rightarrow \text{bit } 1$

$0.25 \times 2 = 0.5 \rightarrow \text{bit } 0$

$0.5 \times 2 = 1.0 \rightarrow \text{bit } 1$

Fraction = 0 → STOP

Binary: 0.101 

2 Fraction becomes periodic (repeating)

- Some decimal fractions **cannot be represented exactly in binary**.
 - The fractional part **repeats infinitely**, forming a pattern.
 - Example: 0.1 in decimal → binary = 0.000110011001100... (pattern 0011 repeats forever)
 - Here, you **stop after desired precision** and round/truncate.
-

Decimal(with fraction) to Binary

Example : 8.125

Step 1: Conversion of 8 to binary

1. $8/2$: Remainder = 0 : Quotient = 4
2. $4/2$: Remainder = 0 : Quotient = 2
3. $2/2$: Remainder = 0 : Quotient = 1
4. $1/2$: Remainder = 1 : Quotient = 0

So equivalent binary of integral part of decimal is 1000.

Step 2: Conversion of .125 to binary

1. $0.125 * 2 = 0.250$, Integral part: 0
2. $0.250 * 2 = 0.500$, Integral part: 0
3. $0.500 * 2 = 1.000$, Integral part: 1

So equivalent binary of fractional part of decimal is .001

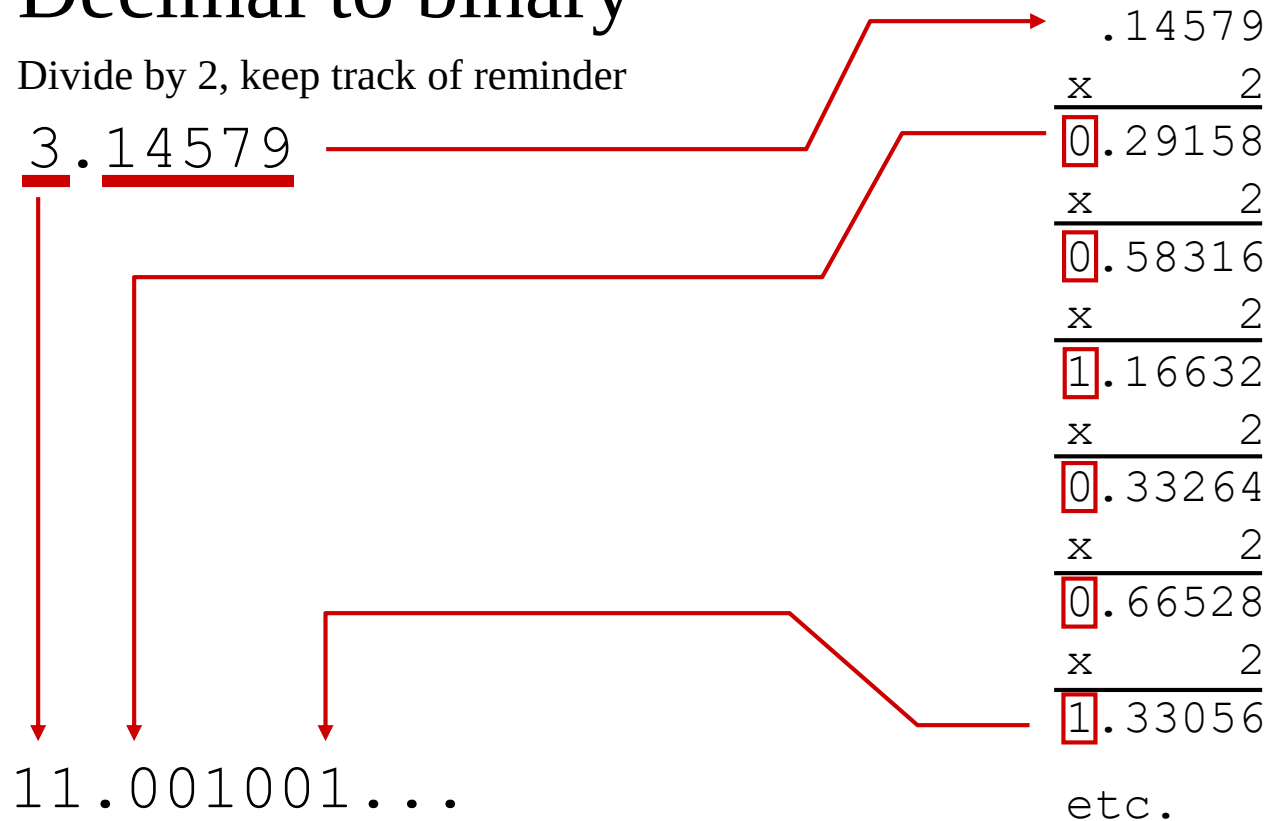
Step 3: Combined the result of step 1 and 2.

Final answer can be written as: 1000.001

Fractions

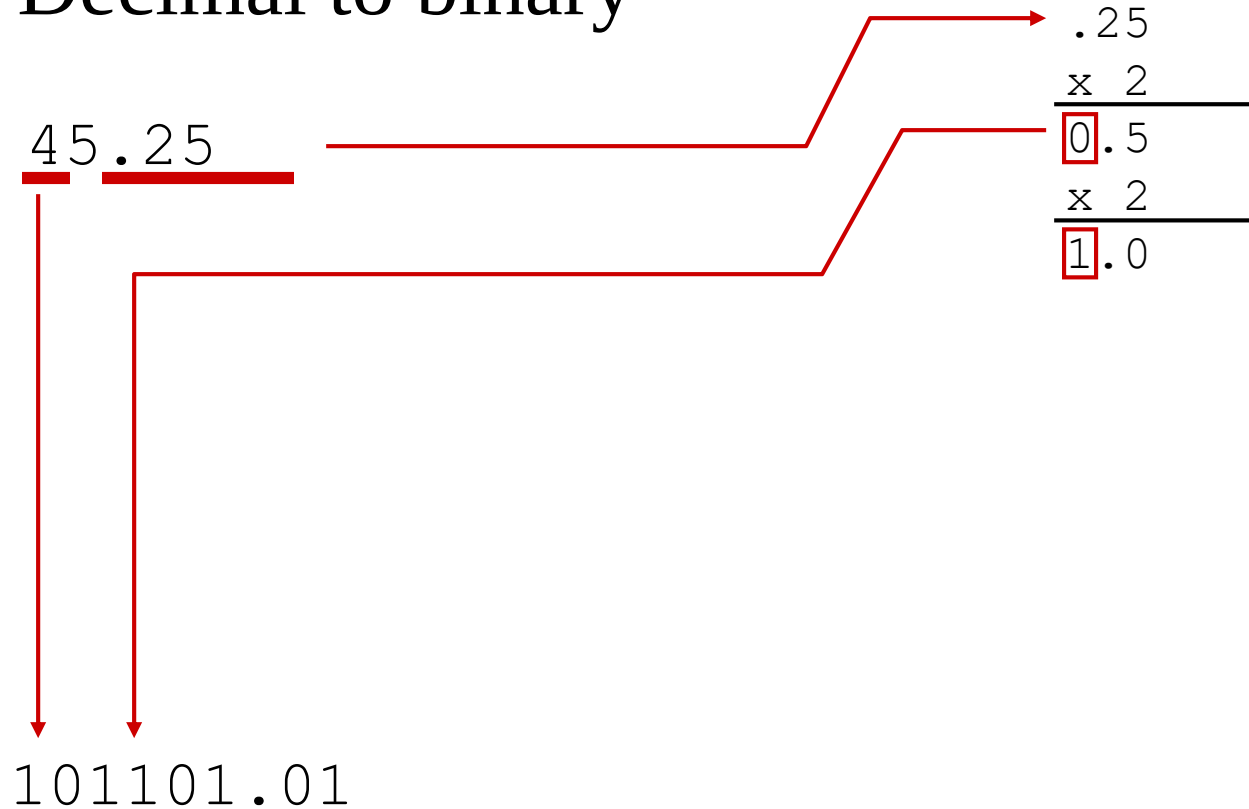
- Decimal to binary

- Divide by 2, keep track of remainder



Fractions

- Decimal to binary

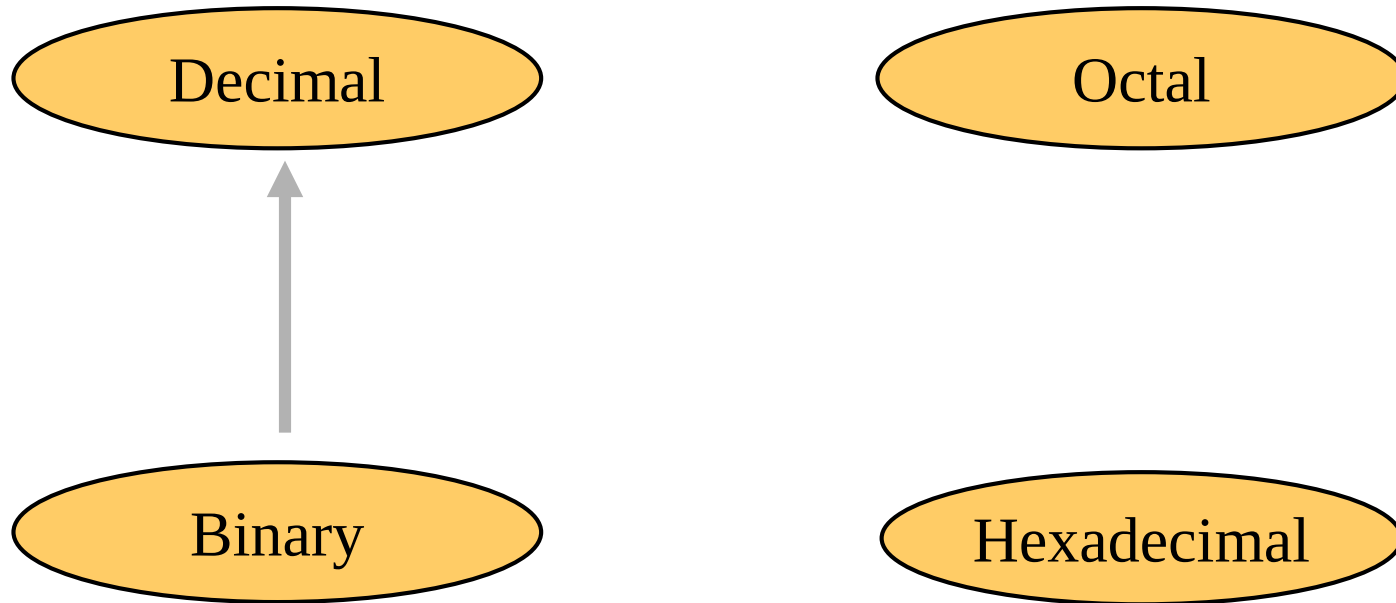


Fractions

- Decimal to binary

$$15.6 = (\mathbf{1111.1001})$$

Binary to Decimal



Binary to Decimal

- Technique
 - Multiply each bit by 2^n , where n is the “weight” of the bit
 - The weight is the position of the bit, starting from 0 on the right
 - Add the results
-

Example

Bit "0"

$101011_2 \Rightarrow$

1	x	2^0	=	1
1	x	2^1	=	2
0	x	2^2	=	0
1	x	2^3	=	8
0	x	2^4	=	0
1	x	2^5	=	32
				43_{10}



Convert 0110101 to decimal.

Given Binary number is 0110101

$$\mathbf{0110101} = (0 \times 2^6) + (1 \times 2^5) + (1 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0)$$

$$= 0 + 32 + 16 + 0 + 4 + 0 + 1$$

$$= 53$$

Therefore, Binary Number 0110101 = 53 Decimal number



Convert the binary number 10100011 to decimal.

Given binary number is 10100011

Using the conversion formula,

$$\begin{aligned}\mathbf{10100011} &= (1 \times 2^7) + (0 \times 2^6) + (1 \times 2^5) + (0 \times 2^4) + (0 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) \\ &= 128 + 0 + 32 + 0 + 0 + 0 + 2 + 1 \\ &= 163\end{aligned}$$

Therefore, binary number 10100011 = 163 decimal number

Convert the binary number 11101111 to decimal.

Given binary number is 11101111

Using the conversion formula,

$$\begin{aligned}\mathbf{11101111} &= (1 \times 2^7) + (1 \times 2^6) + (1 \times 2^5) + (0 \times 2^4) + (1 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) \\ &= 128 + 64 + 32 + 0 + 8 + 4 + 2 + 1 \\ &= 239\end{aligned}$$

Therefore, binary number 11101111 = 239 decimal number

Convert the binary number 1001 to a decimal number.

Given, binary number = 1001_2

Hence, using the binary to decimal conversion formula, we have:

$$\begin{aligned} 1001_2 &= (1 \times 2^3) + (0 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) \\ &= 8 + 0 + 0 + 1 \\ &= (9)_{10} \end{aligned}$$



Convert 1101001_2 into an equivalent decimal number.

Solution: Using binary to decimal conversion method, we get;

$$(1101001)_2 = (1 \times 2^6) + (1 \times 2^5) + (0 \times 2^4) + (1 \times 2^3) + (0 \times 2^2) + (0 \times 2^1) + (1 \times 2^0)$$

$$= 64 + 32 + 0 + 8 + 0 + 0 + 1$$

$$= (105)_{10}$$

Convert $(11110111)_2$ into base-10 number system.

Solution: Using binary to decimal conversion method, we get;

$$(11110111)_2 = (1 \times 2^7) + (1 \times 2^6) + (1 \times 2^5) + (1 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (1 \times 2^0)$$

$$= 128 + 64 + 32 + 16 + 0 + 4 + 2 + 1$$

$$= (247)_{10}$$

Convert $(110.101)_2$ into base-10 number system.

Step 1: Conversion of 110 to decimal

$$\Rightarrow 110_2 = (1 \cdot 2^2) + (1 \cdot 2^1) + (0 \cdot 2^0)$$

$$\Rightarrow 110_2 = 4 + 2 + 0 \Rightarrow 110_2 = 6$$

$\Rightarrow$ So equivalent decimal of binary integral is 6.

Step 2: Conversion of .101 to decimal

$$0.101_2 = (1 \cdot 2^{-1}) + (0 \cdot 2^{-2}) + (1 \cdot 2^{-3})$$

$$\Rightarrow 0.101_2 = (1 \cdot 1/2) + (0 \cdot 1/2^2) + (1 \cdot 1/2^3)$$

$$\Rightarrow 0.101_2 = 1 \cdot 0.5 + 0 \cdot 0.25 + 1 \cdot 0.125$$

$$\Rightarrow 0.101_2 = 0.625$$

$\Rightarrow$ So equivalent decimal of binary fractional is 0.625

Step 3: Add result of step 1 and 2.

$$\Rightarrow 6 + 0.625 = 6.625$$

Convert $(110.101)_2$ into base-10 number system.

Example 2: $(0.11)_2 \rightarrow ?$

Right of point:

$$= 1 \times 2^{-1} + 1 \times 2^{-2} = \frac{1}{2} + \frac{1}{4} = 0.75$$

 $(0.11)_2 = (0.75)_{10}$

Example 3: $(10.01)_2 \rightarrow ?$

Whole part = $10 = (2)_{10}$

Fraction part = $.01 = (0 \times \frac{1}{2} + 1 \times \frac{1}{4}) = 0.25$

 $(10.01)_2 = 2 + 0.25 = 2.25$

Example 4: $(111.011)_2 \rightarrow ?$

Left part: $111 = 7$

Right part: $.011 = (0 \times \frac{1}{2} + 1 \times \frac{1}{4} + 1 \times \frac{1}{8}) = 0.375$

 $(111.011)_2 = 7.375$

Example 5: $(0.1011)_2 \rightarrow ?$

Right part = $1 \times \frac{1}{2} + 0 \times \frac{1}{4} + 1 \times \frac{1}{8} + 1 \times \frac{1}{16}$

$$= 0.5 + 0 + 0.125 + 0.0625 = 0.6875$$

 $(0.1011)_2 = (0.6875)_{10}$

Practice Questions

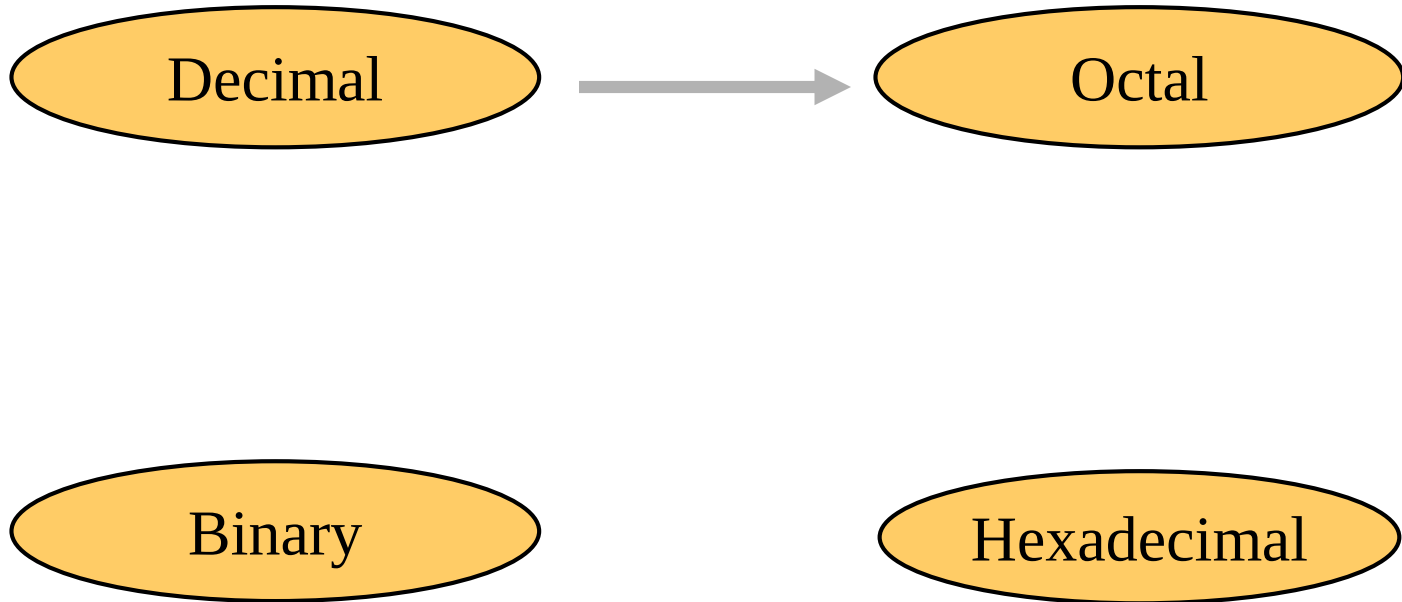
Convert binary number 10111 into an equivalent decimal number.

Convert 111_2 in decimal number.

What is 101010_2 in decimal number?



Decimal to Octal



Decimal to Octal

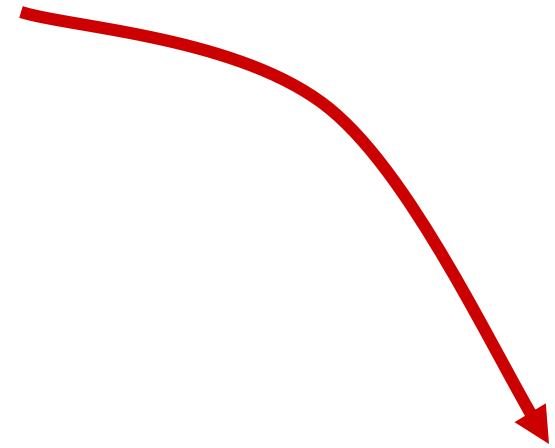
- Technique
 - Divide by 8
 - Keep track of the remainder



Example

$$1234_{10} = ?_8$$

8		1234	
8		154	2
8		19	2
8		2	3
		0	2



$$1234_{10} = 2322_8$$

Example

$$(425)_{10} = (\underline{\quad ? \quad})_8$$

8	425		
8	53	1	↑
8	6	5	↑
	0	6	↑

$$\therefore (425)_{10} = (651)_8$$

Example

$$(6260)_{10} = (\underline{\quad ? \quad})_8$$

8	6260		
8	782	4	↑
8	97	6	↑
8	12	1	↑
8	1	4	↑
	0	1	↑

$$\therefore (6260)_{10} = (14164)_{\underline{8}}$$

Example

Example 1: $(45)_{10} \rightarrow (?)_8$

Step	Division	Quotient	Remainder
$45 \div 8$	5	5	
$5 \div 8$	0	5	

Now read remainders from **bottom to top** $\rightarrow 55$

 $(45)_{10} = (55)_8$

Example 2: $(100)_{10} \rightarrow (?)_8$

Step	Division	Quotient	Remainder
$100 \div 8$	12	4	
$12 \div 8$	1	4	
$1 \div 8$	0	1	

Bottom to top $\rightarrow 144$

 $(100)_{10} = (144)_8$

Example

 Example 3: $(63)_{10} \rightarrow (?)_8$

Step	Division	Quotient	Remainder
$63 \div 8$	7	7	
$7 \div 8$	0	7	

 $(63)_{10} = (77)_8$

 Example 6: $(255)_{10} \rightarrow (?)_8$

Step	Division	Quotient	Remainder
$255 \div 8$	31	7	
$31 \div 8$	3	7	
$3 \div 8$	0	3	

Bottom to top $\rightarrow 377$

 $(255)_{10} = (377)_8$

Example 4: $(78.625)_{10} \rightarrow (?)_8$

Step 1: Integer part $(78)_{10}$

Step	Division	Quotient	Remainder
$78 \div 8$	9	6	
$9 \div 8$	1	1	
$1 \div 8$	0	1	

Bottom to top $\rightarrow 116$

Step 2: Fractional part $(0.625)_{10}$

Step	Multiply by 8	Integer part	Fraction part
$0.625 \times 8 = 5.0$	5	0	

So fractional part = .5

 $(78.625)_{10} = (116.5)_8$

Example 5: $(12.375)_{10} \rightarrow (?)_8$

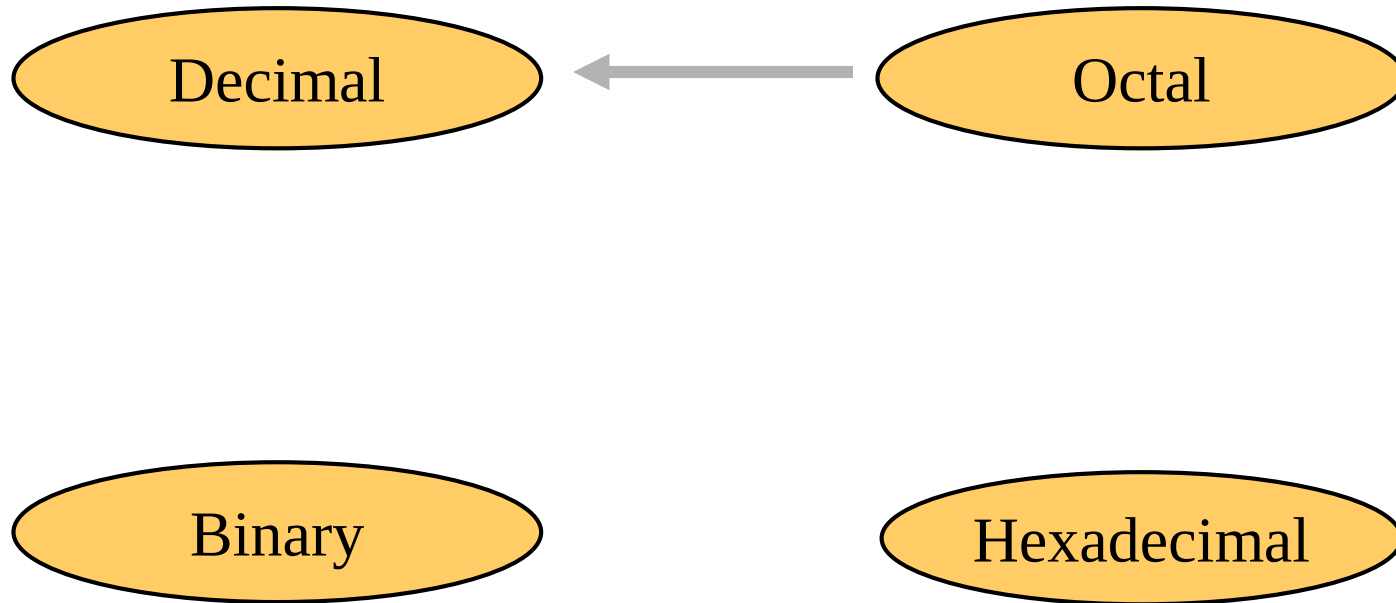
Integer part (12) $\rightarrow 12 \div 8 = 1 \text{ R}4 \rightarrow 1 \div 8 = 0 \text{ R}1 \rightarrow (14)_8$

Fraction part (0.375)

$0.375 \times 8 = 3.0 \rightarrow \text{integer part} = 3$

 $(12.375)_{10} = (14.3)_8$

Octal to Decimal



Octal to Decimal

- Technique
 - Multiply each bit by $\underline{8^n}$, where n is the “weight” of the bit
 - The weight is the position of the bit, starting from 0 on the right
 - Add the results
-

Example

$$\begin{array}{rcll} 724_8 & => & 4 \times 8^0 & = & 4 \\ & & 2 \times 8^1 & = & 16 \\ & & 7 \times 8^2 & = & 448 \\ & & & & \hline & & & & 468_{10} \end{array}$$

Example

- $(304)_8 = (\underline{\quad? \quad})_{10}$

$$= 3 \times 8^2 + 0 \times 8^1 + 4 \times 8^0$$

$$= 3 \times 64 + 0 \times 8 + 4 \times 1$$

$$= 196$$

$$\therefore (304)_8 = (196)_{10}$$

Example

- $(1534)_8 = (\underline{\quad? \quad})_{10}$

$$= 1 \times 8^3 + 5 \times 8^2 + 3 \times 8^1 + 4 \times 8^0$$

$$= 1 \times 512 + 5 \times 64 + 3 \times 8 + 4 \times 1$$

$$= 860$$

$$\therefore (1534)_8 = (860)_{10}$$

Example

Example: Suppose 215_8 is an octal number, then it's decimal form will be,

- $215_8 = 2 \times 8^2 + 1 \times 8^1 + 5 \times 8^0$
- $= 2 \times 64 + 1 \times 8 + 5 \times 1 = 128 + 8 + 5$
- $= 141_{10}$

Example: Let 125 is an octal number denoted by 125_8 . Find the decimal number.

- $125_8 = 1 \times 8^2 + 2 \times 8^1 + 5 \times 8^0$
 - $= 1 \times 64 + 2 \times 8 + 5 \times 1 = 64 + 16 + 5$
 - $= 85_{10}$
 -
-

Convert $(25.4)_8$ to decimal

Step 1: Separate integer and fractional parts

Integer part = 25_8

Fractional part = $.4_8$

Step 2: Convert integer part $(25)_8 \rightarrow$ decimal

$$2 \times 8^1 + 5 \times 8^0 = 16 + 5 = 21$$

Step 3: Convert fractional part $(.4)_8 \rightarrow$ decimal

$$4 \times 8^{-1} = 4 \times \frac{1}{8} = 0.5$$

Step 4: Add both parts

$$21 + 0.5 = 21.5$$

✓ $(25.4)_8 = (21.5)_{10}$

Convert $(7.34)_8$ to decimal

Step 1: Integer part: $7_8 \rightarrow 7 \times 8^0 = 7$

Step 2: Fractional part:

$$3 \times 8^{-1} + 4 \times 8^{-2} = \frac{3}{8} + \frac{4}{64} = 0.375 + 0.0625 = 0.4375$$

Step 3: Add both parts

$$7 + 0.4375 = 7.4375$$

✓ $(7.34)_8 = (7.4375)_{10}$

Convert $(12.65)_8$ to decimal

Integer part:

$$1 \times 8^1 + 2 \times 8^0 = 8 + 2 = 10$$

Fractional part:

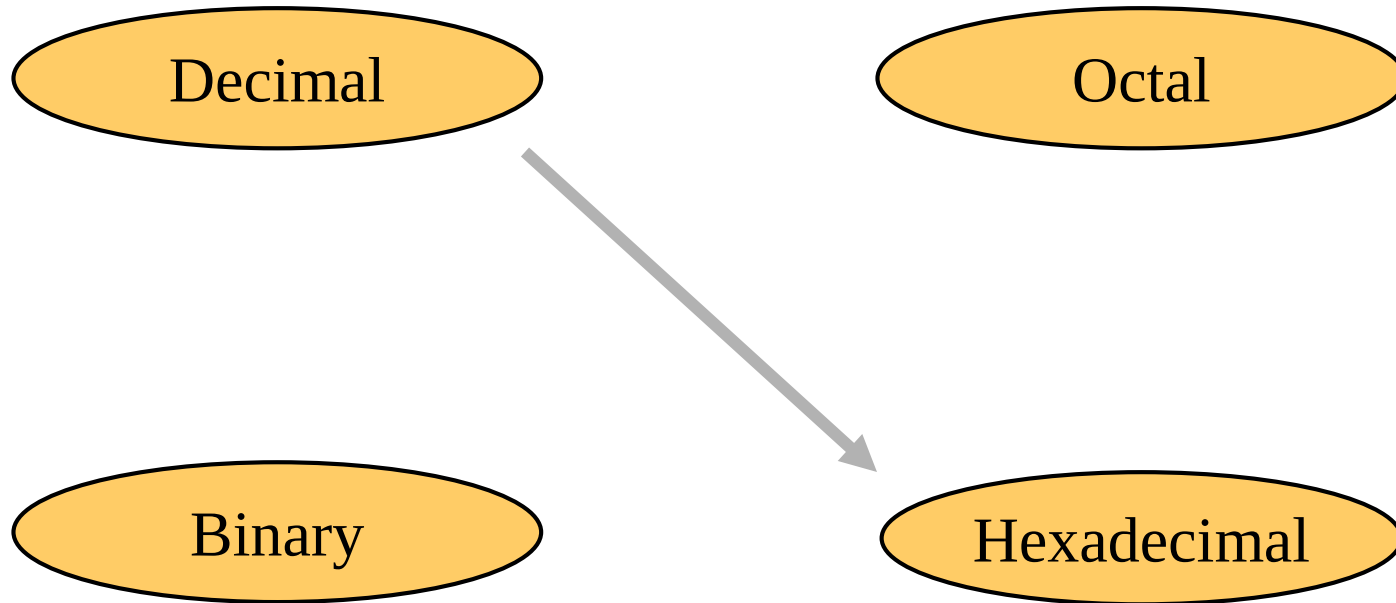
$$6 \times 8^{-1} + 5 \times 8^{-2} = \frac{6}{8} + \frac{5}{64} = 0.75 + 0.078125 = 0.828125$$

Add both parts:

$$10 + 0.828125 = 10.828125$$

✓ $(12.65)_8 = (10.828125)_{10}$

Decimal to Hexadecimal



Decimal to Hexadecimal

- Technique
 - Divide by 16
 - Keep track of the remainder

The hexadecimal (base-16) number system uses 16 symbols:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, *A, B, C, D, E, F*

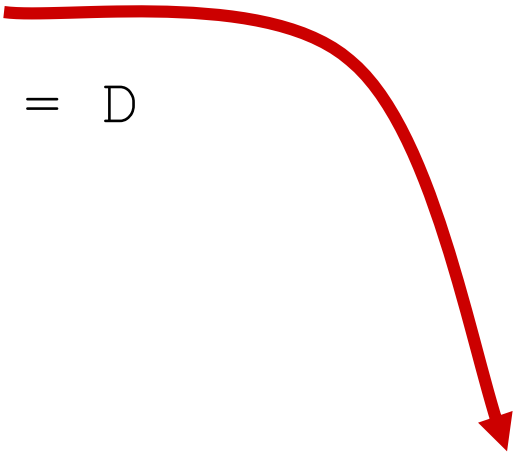
Where:

A = 10, *B* = 11, *C* = 12, *D* = 13, *E* = 14, *F* = 15 (in decimal)

Example

$$1234_{10} = ?_{16}$$

16		1234	
16		77	2
16		4	13 = D
		0	4


$$1234_{10} = 4D2_{16}$$

Example

$$(1423)_{10} = (\underline{\quad ? \quad})_{16}$$

16	1423		↑ ↑ ↑
16	88	F	
16	5	8	
	0	5	

$$\therefore (1423)_{10} = (58F)_{16}$$

Example

$$(93419)_{10} = (\underline{\quad ? \quad})_{16}$$

16	93419		
16	5838	B	↑
16	364	E	↑
16	22	C	↑
16	1	6	↑
	0	1	↑

$$\therefore (93419)_{10} = (16CEB)_{16}$$

Example

Convert 5386 to a hexadecimal number.

Number (Division)	Quotient	Remainder
5386 / 16	336	10 = A
336 / 16	21	0
21 / 16	1	5
1 / 16	0	1

Decimal Value → Hexadecimal Value

$(5386)_{10}$ → $(150A)_{16}$

Example

Convert $(960)_{10}$ into hexadecimal.

$$(960)_{10} = (3C0)_{16}$$

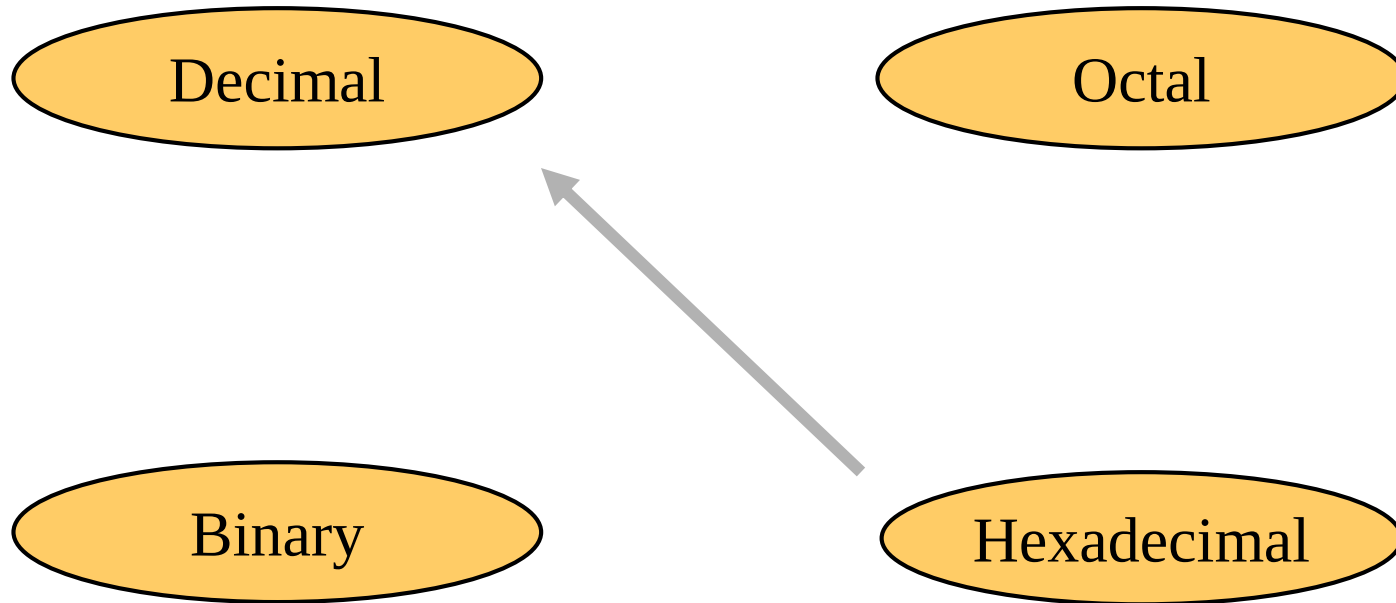
Convert 1228_{10} into hexadecimal.

$$1228_{10} = 4CC_{16}$$

Convert 600_{10} into a hexadecimal number.

$$600_{10} = 258_{16}$$

Hexadecimal to Decimal



Hexadecimal to Decimal

- Technique
 - Multiply each bit by 16^n , where n is the “weight” of the bit
 - The weight is the position of the bit, starting from 0 on the right
 - Add the results
 - Hexadecimal no – 0-9 & A-F
-

Example

$$(A10)_{16} = Q_{10}$$

$$= A \times 16^2 + 1 \times 16^1 + 0 \times 16^0$$

$$= 10 \times 256 + 1 \times 16 + 0 \times 1$$

$$= 2576$$

Example

$$(BCA)_{16} = Q_{10}$$

$$= B \times 16^2 + C \times 16^1 + A \times 16^0$$

$$= 11 \times 256 + 12 \times 16 + 10 \times 1$$

$$= 3018$$

Example

Example 1: What is 5C6 (Hexadecimal)?

- Solution: Step 1: The “5 “ is the “16 x 16” position, so that means $5 \times 16 \times 16$
- Step 2: The ‘C’ (12) is in the “16” position, so that means 12×16 .
- Step 3: The “6” in the “1” position so that means 6.
- Answer is : $5C6 = 5 \times 16 \times 16 + 12 \times 16 + 6 = (1478)$ in Decimal.

Example 2: What is 3C5 (Hexadecimal)?

- Solution: Step 1: The “3 “ is the “16 x 16” position, so that means $3 \times 16 \times 16$
 - Step 2: The ‘C’ (12) is in the “16” position, so that means 12×16 .
 - Step 3: The “5” is in the “1” position so that means 5.
 - Answer is : $3C5 = 3 \times 16 \times 16 + 12 \times 16 + 5 = (965)$ in Decimal.
-

Example

Example 3: What is 7B5 (Hexadecimal)?

- Solution: Step 1: The “7 “ is the “16 x 16” position, so that means $7 \times 16 \times 16$
- Step 2: The ‘B’ (11) is in the “16” position, so that means 11×16 .
- Step 3: The 5” in the “1” position so that means 5.
- Answer is : $7B5 = 7 \times 16 \times 16 + 11 \times 16 + 5 = (1973)$ in Decimal.

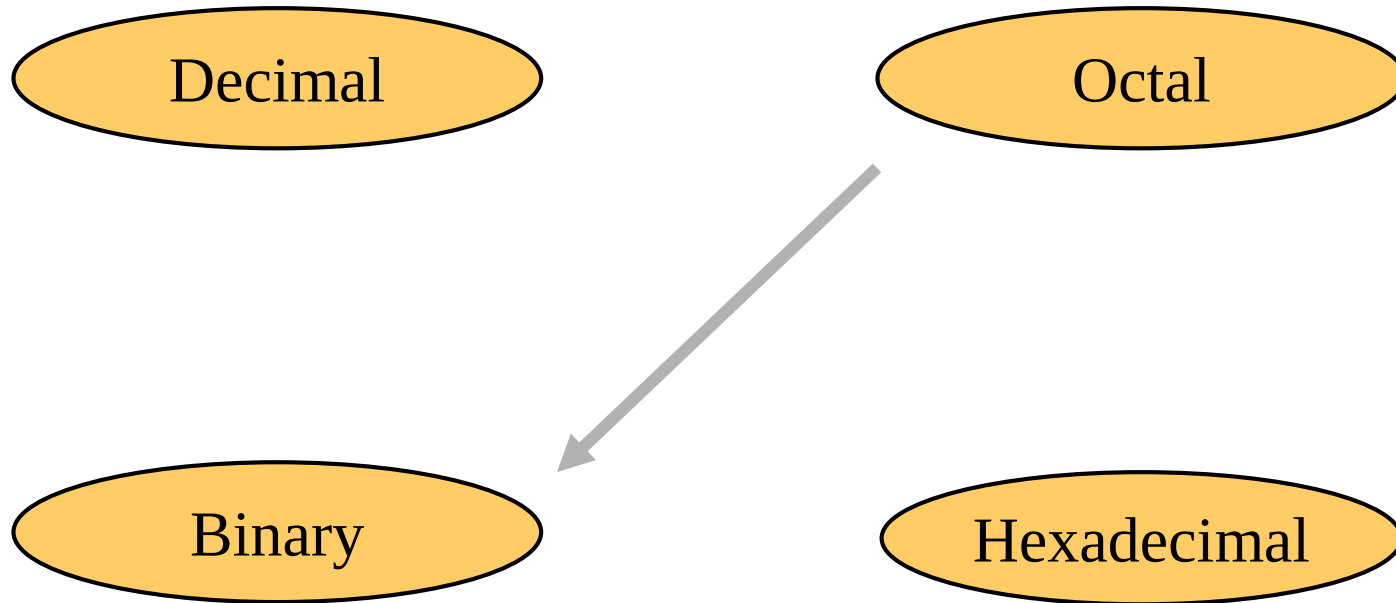
Example 4: What is 2E8 (Hexadecimal)?

- Solution: Step 1: The “2 “ is the “16 x 16” position, so that means $2 \times 16 \times 16$
- Step 2: The ‘E’ (14) is in the “16” position, so that means 14×16 .
- Step 3: The “2” is in the “1” position so that means 2.
- Answer is : $2E8 = 2 \times 16 \times 16 + 14 \times 16 + 8 = (744)$ in Decimal.

Example 5: What is 4F8 (Hexadecimal)?

- Solution: Step 1: The “4 “ is the “16 x 16” position, so that means $4 \times 16 \times 16$
 - Step 2: The ‘F’ (15) is in the “16” position, so that means 15×16 .
 - Step 3: The “8” is in the “1” position, which means 8.
 - Answer is : $4F8 = 4 \times 16 \times 16 + 15 \times 16 + 8 = (1272)$ in Decimal.
-

Octal to Binary



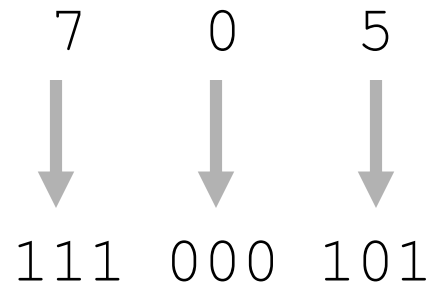
Octal to Binary

- Technique
 - Convert each octal digit to a 3-bit equivalent binary representation



Example

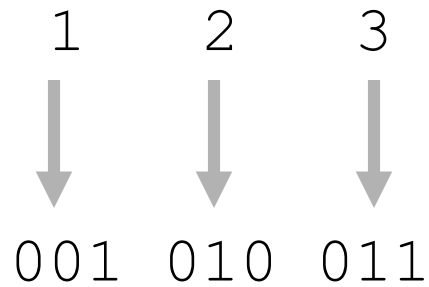
$$705_8 = ?_2$$



$$705_8 = 111000101_2$$

Example

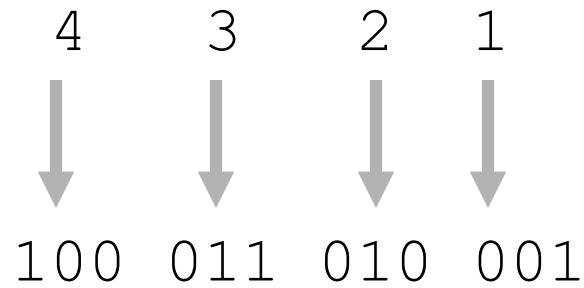
$$(123)_8 = (\underline{\quad? \quad})_2$$



$$\therefore (123)_8 = (1010011)_2$$

Example

$$(4321)_8 = (\underline{\quad? \quad})_2$$



$$\therefore (4321)_8 = (100011010001)_2$$

Example 4 (with fraction)

Octal: 12.5_8

Convert both parts separately 

Octal	Binary
1	001
2	010
.	.
5	101

✓ Binary: 001010.101_2

(or simply 1010.101_2 after removing leading zeros)

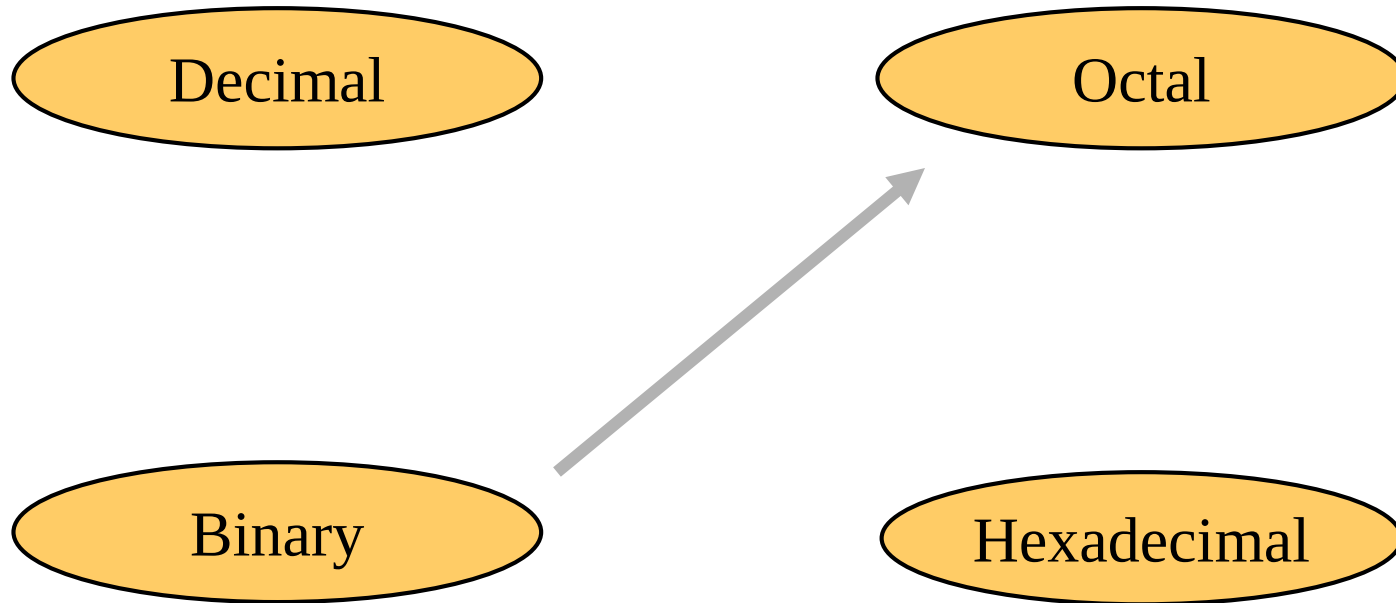
Example 5

Octal: 7.32_8

Octal	Binary
7	111
.	.
3	011
2	010

✓ Binary: 111.011010_2

Binary to Octal



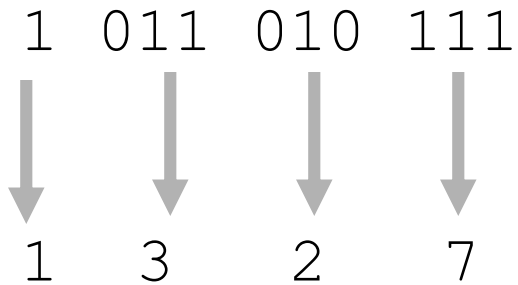
Binary to Octal

- Technique
 - Group bits in threes, starting on right
 - Convert to octal digits



Example

$$1011010111_2 = ?_8$$



$$1011010111_2 = 1327_8$$

Example

$$(11011001)_2 = (\underline{\quad ? \quad})_8$$

$$\frac{011}{3}$$

$$\frac{011}{3}$$

$$\frac{001}{1}$$

$$\therefore (11011001)_2 = (331)_8$$

Example

$$(10110011)_2 = (\underline{\quad ? \quad})_8$$

010

2

110

6

011

3

$$\therefore (10110011)_2 = (263)_8$$

Example

Example: Convert binary number 1010111100 into octal number.

Therefore, Binary to octal is.

= (1010111100)

= (001 010 111 100)

= (1 2 7 4)

= (1274)

Example 1

Binary: 110010

Step 1: Group into 3 bits from right:

110 010

Step 2: Convert each group:

110 → 6

010 → 2

✓ Octal: 62₈

Example 2

Binary: 1010111

Step 1: Group from right (add leading zeros if needed):

001 010 111

Step 2: Convert each:

001 → 1

010 → 2

111 → 7

✓ Octal: 127₈

Example 2:

Binary: 111010.011

Step 1:

- Integer: 111010 → 111 010
- Fraction: .011

Step 2: Convert each

111 → 7

010 → 2

.011 → .3

✓ Octal: 72.3₈

Example 3:

Binary: 101011.1101

Step 1:

- Integer: 101011 → 101 011
- Fraction: .1101 → .110 100 (pad with one zero)

Step 2: Convert

101 → 5

011 → 3

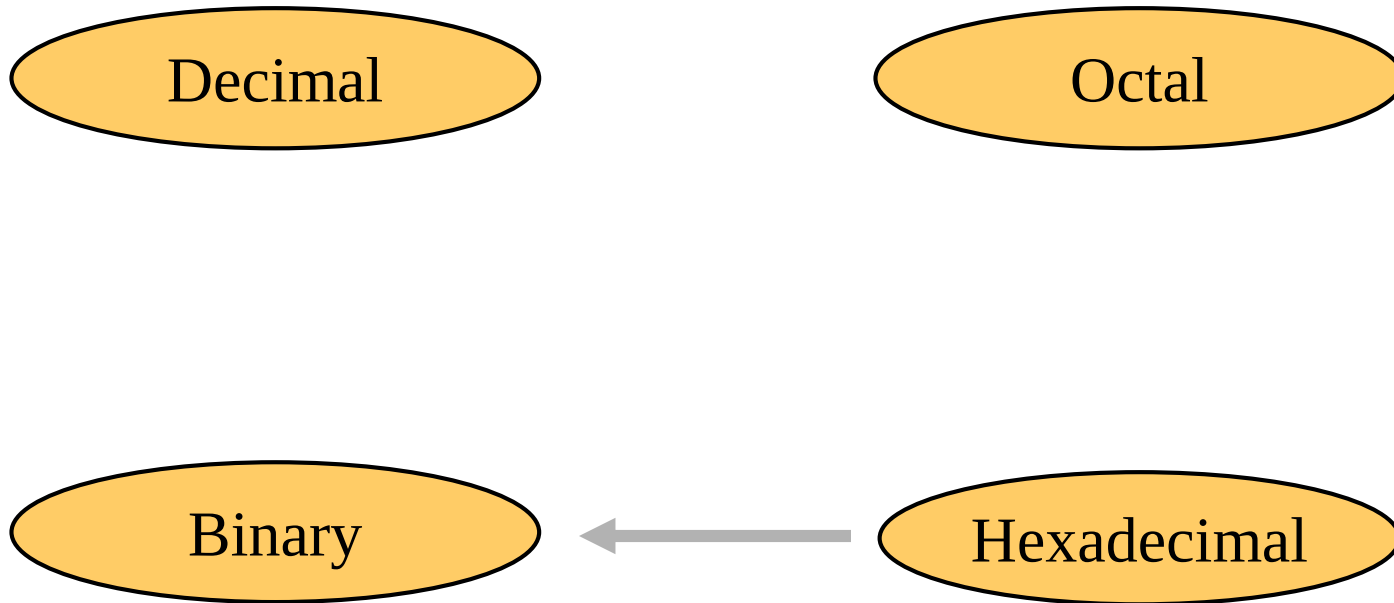
.110 → .6

.100 → .4

Combine fractional digits → .64

✓ Octal: 53.64₈

Hexadecimal to Binary



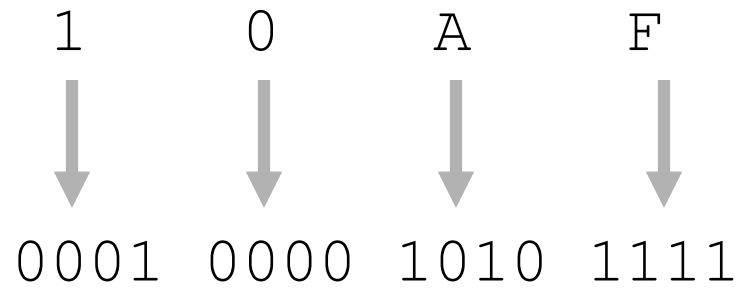
Hexadecimal to Binary

- Technique
 - Convert each hexadecimal digit to a 4-bit equivalent binary representation



Example

$$10AF_{16} = ?_2$$



$$10AF_{16} = 0001000010101111_2$$

Example

$$(283)_{16} = (\underline{\quad ? \quad})_2$$

2

0010

8

1000

3

0011

$$\therefore (283)_{16} = (1010000011)_2$$



Example 2:

Convert $(A3)_{16} \rightarrow$ Binary

$$A = 1010, \quad 3 = 0011$$

So,

$$(A3)_{16} = 1010\ 0011_2$$

 $(A3)_{16} = (10100011)_2$

Example 3:

Convert $(7B4)_{16} \rightarrow$ Binary

$$7 = 0111, \quad B = 1011, \quad 4 = 0100$$

Combine:

$$(7B4)_{16} = 0111\ 1011\ 0100_2$$

 $(7B4)_{16} = (011110110100)_2$

Example 4 (with fraction):

Convert $(3A.F)_{16} \rightarrow$ Binary

$$3 = 0011, \quad A = 1010, \quad F = 1111$$

So,

$$(3A.F)_{16} = 0011\ 1010.1111_2$$

 $(3A.F)_{16} = (00111010.1111)_2$

Example 5:

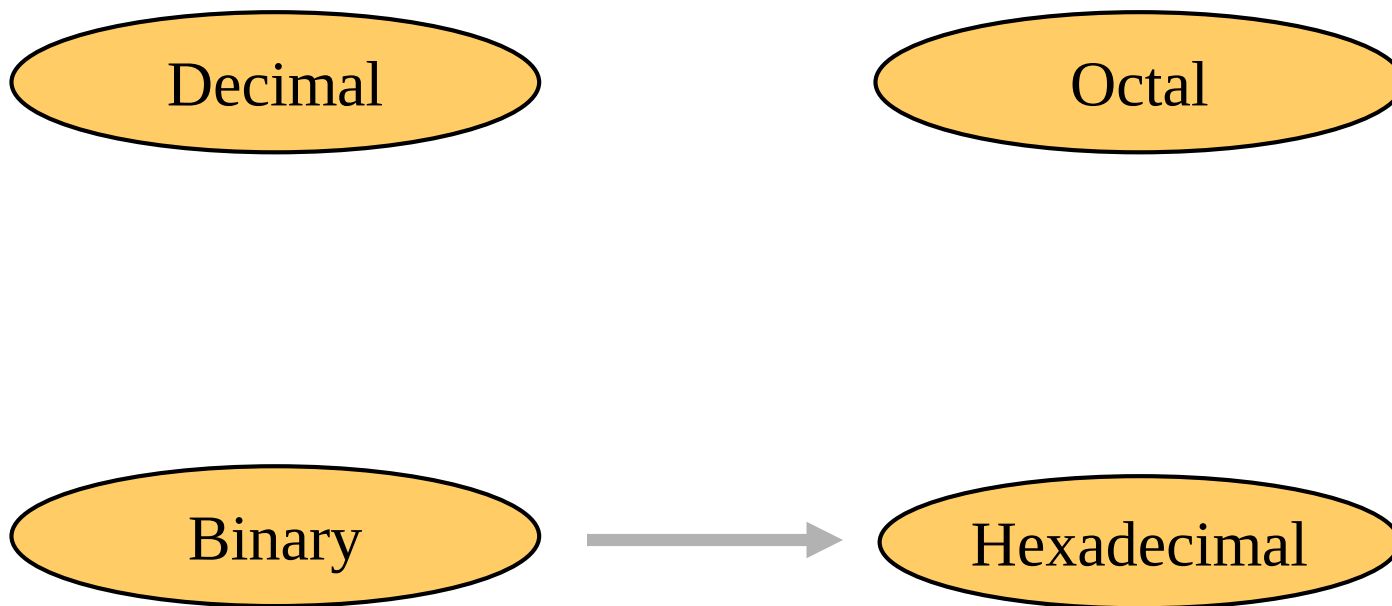
Convert $(1C9)_{16} \rightarrow$ Binary

$$1 = 0001, \quad C = 1100, \quad 9 = 1001$$

$$(1C9)_{16} = 0001\ 1100\ 1001_2$$

 $(1C9)_{16} = (000111001001)_2$

Binary to Hexadecimal



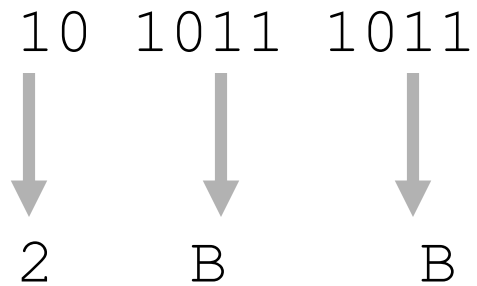
Binary to Hexadecimal

- Technique
 - Group bits in fours, starting on right
 - Convert to hexadecimal digits



Example

$$1010111011_2 = ?_{16}$$



$$1010111011_2 = 2BB_{16}$$

Example

$$(1111010100)_2 = (\underline{\quad? \quad})_{16}$$

0011

3

1101

D

0100

4

$$\therefore (1111010100)_2 = (3D\cancel{4})_{16}$$



Example

$$2. (1111010100111110)_2 = (\underline{\quad ? \quad})_{16}$$

Solution:

$$(1111010100111110)_2 = (\underline{\quad})_{16}$$

$$\begin{array}{cccc} \underline{1111} & \underline{0101} & \underline{0011} & \underline{1110} \\ F & 5 & 3 & E \end{array}$$

$$\therefore (1111010100111110)_2 = (\underline{F53E})_{16}$$

Example 1:

Convert $(10101111)_2 \rightarrow$ Hex

Group into 4 bits (from right):

1010 1111

Now convert each group:

$$1010 = A, \quad 1111 = F$$

 $(10101111)_2 = (AF)_{16}$

Example 2:

Convert $(110110100)_2 \rightarrow$ Hex

Add leading zeros to make groups of 4 bits:

0001 1011 0100

Now convert:

$$0001 = 1, \quad 1011 = B, \quad 0100 = 4$$

 $(110110100)_2 = (1B4)_{16}$

Example 3:

Convert $(11110011.1011)_2 \rightarrow \text{Hex}$

Split integer and fractional parts:

- Integer: 11110011 $\rightarrow$ group as 1111 0011
- Fractional: .1011 $\rightarrow$ group as .1011

Convert:

$$1111 = F, \quad 0011 = 3, \quad .1011 = .B$$

 $(11110011.1011)_2 = (F3.B)_{16}$

Example 4:

Convert $(100111010101)_2 \rightarrow \text{Hex}$

Group into 4 bits:

$$0001 \ 0011 \ 0101 \ 0101$$

Convert each group:

$$0001 = 1, \ 0011 = 3, \ 0101 = 5, \ 0101 = 5$$

 $(100111010101)_2 = (1355)_{16}$

Example 4

Convert $(1011110.101011)_2 \rightarrow \text{Hex}$

Step 1: Group

Integer part: 1011110 $\rightarrow$ 0101 1110

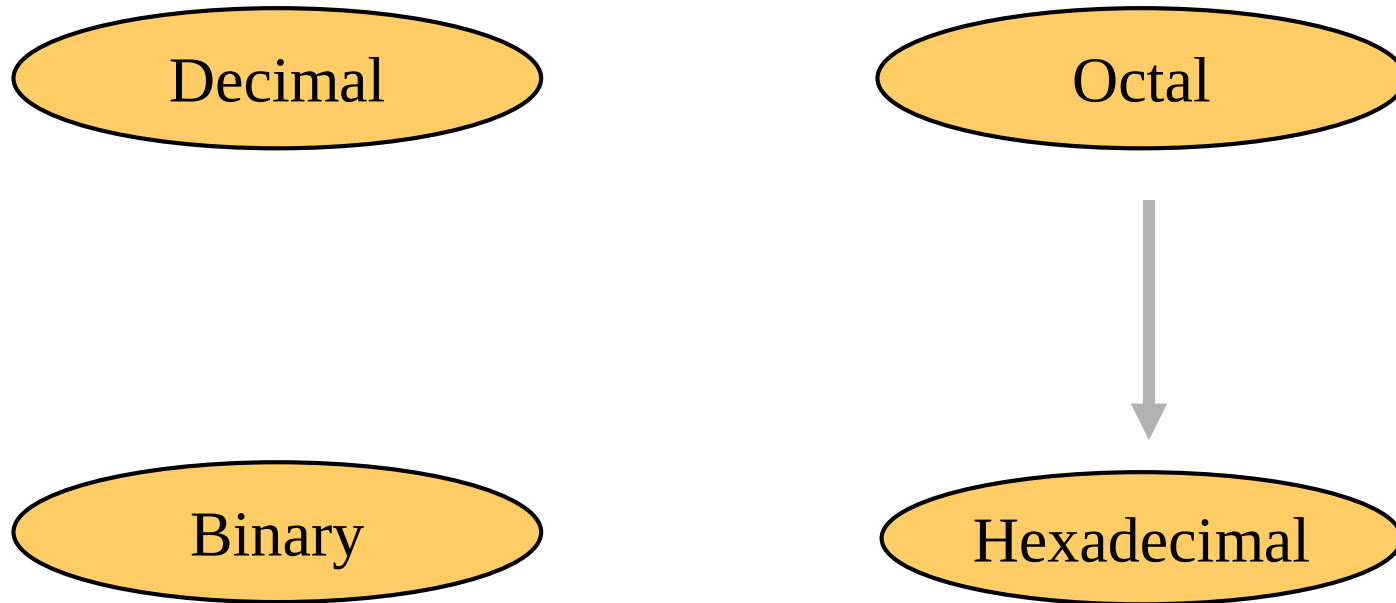
Fractional part: .101011 $\rightarrow$.1010 1100 (pad 0s if needed)

Step 2: Convert

0101 = 5, 1110 = E, .1010 = A, 1100 = C

 $(1011110.101011)_2 = (5E.AC)_{16}$

Octal to Hexadecimal



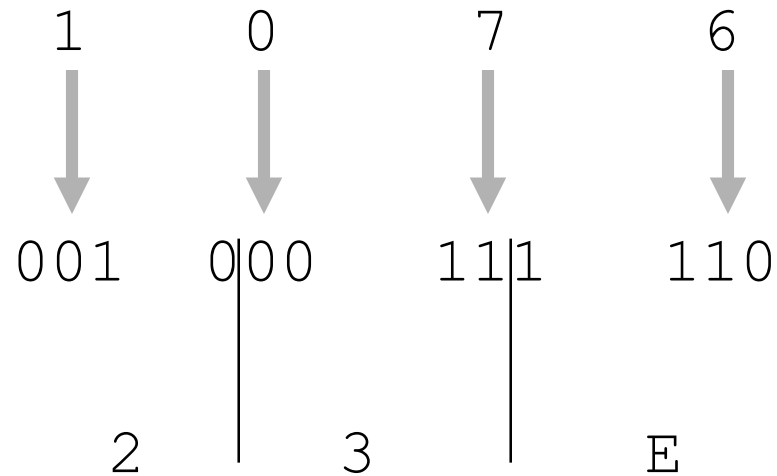
Octal to Hexadecimal

- Technique
 - Use binary as an intermediary
 - Octal $\rightarrow$ binary then binary $\rightarrow$ hexadecimal



Example

$$1076_8 = ?_{16}$$



$$1076_8 = 23E_{16}$$

Example

$$(567)_8 = (\underline{\quad ? \quad})_{16}$$

First convert octal to binary

$$(567)_8 = (\underline{\quad})_2$$

5

101

6

110

7

111

$$\therefore (567)_8 = (101110111)_2$$

Now convert binary to hexadecimal

$$(101110111)_2 = (\underline{\quad})_{16}$$

0001

1

0111

7

0111

7

$$\therefore (101110111)_2 = (177)_{16}$$

$$\therefore (567)_8 = (177)_{16}$$

Example

$$(4321)_8 = (\underline{\quad? \quad})_{16}$$

First convert octal to binary

$$(4321)_8 = ()_2$$

4	3	2	1
<u>100</u>	<u>011</u>	<u>010</u>	<u>001</u>

$$\therefore (4321)_8 = (100011010001)_2$$

Now convert binary to hexadecimal

$$(100011010001)_2 = ()_{16}$$

<u>1000</u>	<u>1101</u>	<u>0001</u>
8	D	1

$$\therefore (100011010001)_2 = (8D1)_{16}$$

$$\therefore (4321)_8 = (8D1)_{16}$$

Example

Convert 536 from octal to hexadecimal number

Convert 536(octal) into its binary equivalent we get

$$(536)_8 = (101) (011) (110)$$

$$=(101011110)_2$$

Now forming the group of 4 binary bits to obtain its hexadecimal equivalent,

$$(101011110)_2 = (0001) (0101) (1110)$$

$$= (15E)_{16}$$

So the hexadecimal number of 536 is 15E.

Example

Convert 752 from octal to hexadecimal number

Step 1:

Octal to Binary Conversion

7	5	2
111	101	010

So the binary equivalent is 111101010

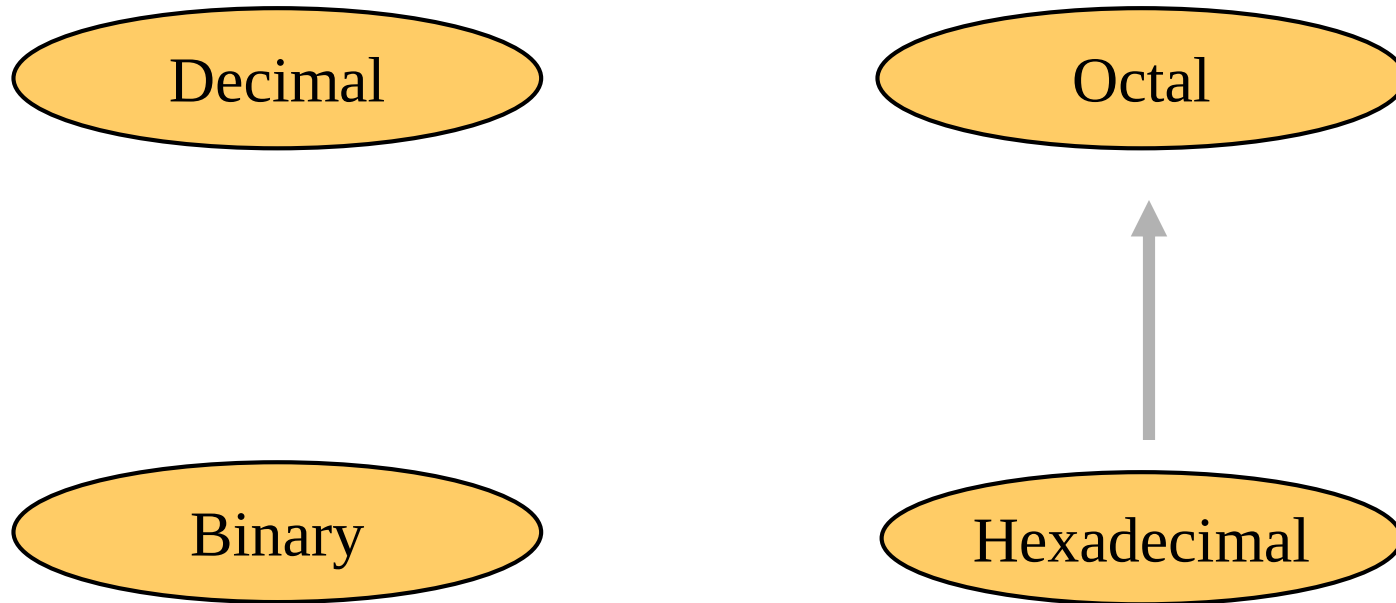
Step 2:

Binary to Hex Conversion

<u>0001</u>	<u>1110</u>	<u>1010</u>
1	D	9



Hexadecimal to Octal



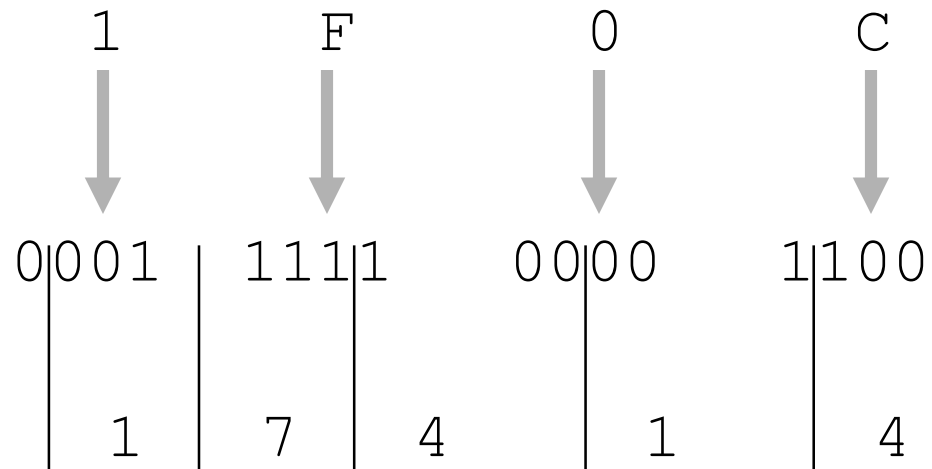
Hexadecimal to Octal

- Technique
 - Use binary as an intermediary



Example

$$1F0C_{16} = ?_8$$



$$1F0C_{16} = 17414_8$$

Example

Example: Convert $1BC_{16}$ into an octal number.

Solution: Given, $1BC_{16}$ is a hexadecimal number.

$1 \rightarrow 0001$, $B \rightarrow 1011$, $C \rightarrow 1100$

Now group them from right to left, each having 3 digits.

000, 110, 111, 100

$000 \rightarrow 0$, $110 \rightarrow 6$, $111 \rightarrow 7$, $100 \rightarrow 4$

Hence, $1BC_{16} = 674_8$

Example

$$(951)_{16} = (\underline{\quad? \quad})_8$$

First convert hexadecimal to binary

$$(951)_{16} = (\underline{\quad})_2$$

9

1001

5

0101

1

0001

$$\therefore (951)_{16} = (100101010001)_2$$

Now convert binary to octal

$$(100101010001)_2 = (\underline{\quad})_8$$

100

4

101

5

010

2

001

1

$$\therefore (100101010001)_2 = (4521)_8$$

$$\therefore (951)_{16} = (4521)_8$$

Example

$$(FC3A)_{16} = (\underline{\quad? \quad})_8$$

First convert hexadecimal to binary

$$(FC3A)_{16} = Q_2$$

F	C	3	A
<u>1111</u>	<u>1100</u>	<u>0011</u>	<u>1010</u>

$$\therefore (FC3A)_{16} = (1111110000111010)_2$$

Now convert binary to octal

$$(1111110000111010)_2 = Q_8$$

<u>001</u>	<u>111</u>	<u>110</u>	<u>000</u>	<u>111</u>	<u>010</u>
1	7	6	0	7	2

$$\therefore (1111110000111010)_2 = (176072)_8$$

$$\therefore (FC3A)_{16} = (176072)_8$$

Example

Convert the following hexadecimal number to octal number $2CD_{16}$.

Given, $2CD_{16}$ is a hexadecimal number.

$2 \rightarrow 0010$, $C \rightarrow 1100$, $D \rightarrow 1101$,

Now you will be grouping them from right to left, each having 3 digits.

001, 011, 001, 101

$001 \rightarrow 1$, $011 \rightarrow 3$, $001 \rightarrow 1$, $101 \rightarrow 5$

Hence, $2CD_{16} = 1315_8$

Example

Convert the following hexadecimal number to octal number $3EC_{16}$.

$3EC_{16}$ is a hexadecimal number.

$3 \rightarrow 0010$, $E \rightarrow 1110$, $C \rightarrow 1100$,

Now you will be grouping them from right to left, each having 3 digits.

001, 011, 101, 100

$001 \rightarrow 1$, $011 \rightarrow 3$, $101 \rightarrow 5$, $100 \rightarrow 4$

Hence, $3EC_{16} = 1354_8$

Example

Convert $ABCD_{16}$ to equivalent octal form.

Convert 912_{16} to equivalent octal form.



Binary Addition (1 of 2)

- **What is Binary Addition**
 - The binary addition operation works similarly to the base 10 decimal system, except that it is a base 2 system.
 - The binary system consists of only two digits, 1 and 0.
 - Most of the functionalities of the computer system use the binary number system.
 - The binary code uses the digits 1's and 0's to make certain processes turn off or on.
 - The process of the addition operation is very familiar to the decimal system by adjusting to the base 2.
 - Before attempting the binary addition process, we should have complete knowledge of how the place works in the binary number system.
 - Because most of the modern digital computers and electronic circuits perform the binary operation by representing each bit as a voltage signal.
 - The bit 0 represents the “OFF” state, and the bit 1 represents the “ON” state.
-

Rules of Binary Addition

- $0 + 0 = 0$
 - $0 + 1 = 1$
 - $1 + 0 = 1$
 - $1 + 1 = 10$ (carry 1 to the next significant bit)
 - $1 + 1 + 1 = 11$ (carry 1 to the next significant bit)
-

Binary Addition

Weight	16	8	4	2	1
		1	0	1	0
+		1	1	1	0
	1	1	0	0	0

$(10)_{10}$

$(14)_{10}$

$(24)_{10}$

Binary Addition

A binary addition diagram on a dark background. The title "Binary Addition" is at the top. The diagram shows the addition of two binary numbers: 100101.0 and 0100110. The first number is split into integer (100101) and fractional (.0) parts by a vertical dashed line. The second number is 0100110. A carry sequence of 1 1 1 is shown above the integer part of the first number. A plus sign is to the left of the second number. A horizontal line separates the addends from the result. The result is 1110.01, also split into integer (1110) and fractional (.01) parts by a vertical dashed line.

Carry				1	1	1			
		1	0	0	1	.	0	1	1
+		0	1	0	0	.	1	1	0
		1	1	1	0	.	0	0	1

Binary Addition

$$101_2 + 10_2 =$$

1 0 1

+ 1 0

.....

1 1 1



Binary Addition

$$1001_2 + 111_2 =$$

$$\begin{array}{r} 111 \\ 1001 \\ + 111 \\ \hline 10000 \end{array}$$



Binary Addition

$$1010_2 + 1111_2 =$$

$$\begin{array}{r} 1 1 0 \\ 1 0 1 0 \\ + 1 1 1 1 \\ \hline 1 1 0 0 1 \end{array}$$



Binary Addition

Add: 10011_2 and 110001_2

$$\begin{array}{r} 1 1 1 \\ 1 0 0 1 1 \\ + 1 1 0 0 0 1 \\ \hline 1 0 0 0 1 0 0 \end{array}$$

Binary Addition

Add: 1 + 11 (100)

1010 + 11 (1101)

100101 + 10101 (111010)



Binary subtraction

- Binary subtraction is one of the four binary operations, where we perform the subtraction method for two binary numbers (comprising only two digits, 0 and 1).
 - This operation is similar to the basic arithmetic subtraction performed on decimal numbers in Maths.
 - Hence, when we subtract 1 from 0, we need to borrow 1 from the next higher order digit, to reduce the digit by 1 and the remainder left here is also 1.
-

Binary Subtraction Rules

Binary Number	Subtraction Value
$0 - 0$	0
$1 - 0$	1
$0 - 1$	1 (Borrow 1 from the next high order digit)
$1 - 1$	0

Binary Subtraction Examples

$$\begin{array}{r} 1 \ 1 \ 1 \ 1 \\ - \quad 1 \ 0 \ 0 \\ \hline 1 \ 0 \ 1 \ 1 \\ \hline \end{array}$$

Binary Subtraction Examples

Example : $1010 - 101$

1 1 borrow

$$\begin{array}{r} 1010 \\ - 101 \\ \hline \end{array}$$

0101

Binary Subtraction Examples

Example : $1001 - 101$

1 borrow

$$\begin{array}{r} 1001 \\ - 101 \\ \hline 0100 \end{array}$$



Binary Subtraction Examples

Example : Subtract 1101_2 from 10110_2 .

1 1 borrow

$$\begin{array}{r} 10110 \\ - 1101 \\ \hline 01001 \end{array}$$



Binary Subtraction

$$1011011 - 10010 = 1001001$$

$$1010110 - 101010 = 101100$$

$$101101 - 100111 = 110$$

$$100010110 - 1111010 = 10011100$$

$$1000101 - 101100 = 11001$$

$$1110110 - 1010111 = 11111$$

◆ 1. Negative Numbers in Computers

- Computers naturally store numbers in **binary**, which represents only **positive integers**.
- To represent **negative numbers**, special techniques called **complement systems** are used.

◆ 2. Use of Complements

Complement = a way of representing negative numbers such that **subtraction can be performed as addition**.

Types of Complements

Number System	r's Complement	(r-1)'s Complement
Decimal (base 10)	10's complement	9's complement
Binary (base 2)	2's complement	1's complement
Octal (base 8)	8's complement	7's complement
Hexadecimal (base 16)	16's complement	15's complement

r = base of the number system

Idea:

- Subtracting a number = adding its complement.
 - Makes arithmetic simpler in digital computers.
-

◆ What are r 's and $(r - 1)$'s Complements?

These are **general forms of complements** used in **any number system** — not just decimal.

Number System	Base (r)	$(r - 1)$'s Complement	r 's Complement
Decimal	10	9's complement	10's complement
Binary	2	1's complement	2's complement
Octal	8	7's complement	8's complement
Hexadecimal	16	15's complement	16's complement

So basically:

- $(r - 1)$'s complement = Subtract each digit from $(r - 1)$.
- r 's complement = $((r - 1)$'s complement) + 1

◆ Example in Decimal ($r = 10$)

Let's take the number 3254.

- $(r - 1)$'s complement = 9's complement = $9999 - 3254 = 6745$
 - r 's complement = 10's complement = $9999 - 3254 + 1 = 6746$
-

◆ Example in Binary ($r = 2$)

Let's take 1010_2 (which is 10 in decimal).

- $(r - 1)$'s complement $\rightarrow$ 1's complement $\rightarrow$ flip all bits
 $1010 \rightarrow 0101$
- r 's complement $\rightarrow$ 2's complement $\rightarrow 0101 + 1 = 0110$

✓ 2's complement = 0110_2

◆ Why Complements Are Used in Computers

Complements make subtraction easier and allow the computer to handle negative numbers using addition only.

➤ 1 Simplify Subtraction

Instead of building separate circuits for subtraction, computers:

- Take the complement of a number
- Add it to the other number
- Discard the carry (if any)

This way, subtraction is performed using addition hardware only.

➤ 2 Represent Negative Numbers

In computers, especially in **binary**, negative numbers are represented using **2's complement** form. This makes arithmetic (addition/subtraction) consistent and simple.

For example:

Decimal	Binary (8-bit 2's complement)
+5	0000101
-5	11111011

◆ 3. Complements in Binary

A) 1's Complement

- Invert all bits of the number.
- Represents **negative numbers** as 1's complement of the positive number.

Example:

+5 → 0101 (4-bit binary)

-5 → invert all bits → 1010

Note:

- 1's complement has **two zeros** (0000 and 1111)
 - Less common in modern computers.
-

B) 2's Complement

- Take 1's complement of the number and **add 1** to the LSB.
- Represents negative numbers in a way that allows **direct binary addition/subtraction**.

Example:

+5 → 0101

Step 1: 1's complement → 1010

Step 2: Add 1 → 1011

✓ -5 = 1011 (in 4-bit 2's complement)

Advantages of 2's complement:

- Only **one zero**
 - Subtraction becomes addition
 - Widely used in computers
-

◆ 4. Using Complements to Perform Subtraction

Example: Compute $7 - 5$

1 Represent 7 and -5 in 4-bit 2's complement:

- $7 \rightarrow 0111$
- $5 \rightarrow 0101 \rightarrow 2\text{'s complement} \rightarrow 1011$

2 Add them:

markdown

```
  0111
+ 1011
-----
 0010  (ignore overflow)
```

✓ Result = 2 ✓

Subtraction is now **just addition** of the complement of the number.

◆ 5. Decimal Example with 10's Complement

Compute $725 - 348$ using 10's complement:

1 Take 10's complement of 348:

- 9's complement: $999 - 348 = 651$
- 10's complement: $651 + 1 = 652$

2 Add to 725:

markdown

```
  725
+ 652
-----
1377
```

3 Ignore leftmost carry $\rightarrow 377$

✓ $725 - 348 = 377$ ✓

◆ 6. Summary Table

Concept	Binary	Decimal
1's complement	Invert bits	9's complement
2's complement	Invert bits + 1	10's complement
Negative numbers	Represented using complements	Subtract using addition

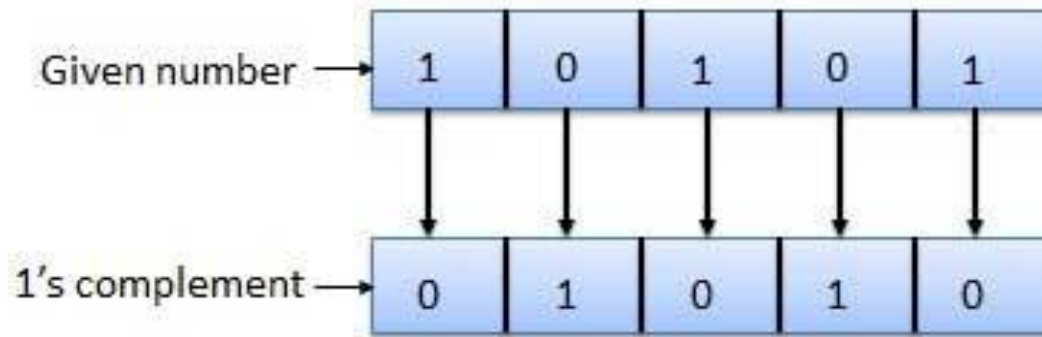
Complements

- Complements are used in the digital computers in order to simplify the subtraction operation and for the logical manipulations.
- For each radix-r system (radix r represents base of number system) there are two types of complements.

S.N.	Complement	Description
1	Radix Complement	The radix complement is referred to as the r 's complement
2	Diminished Radix Complement	The diminished radix complement is referred to as the $(r-1)$'s complement

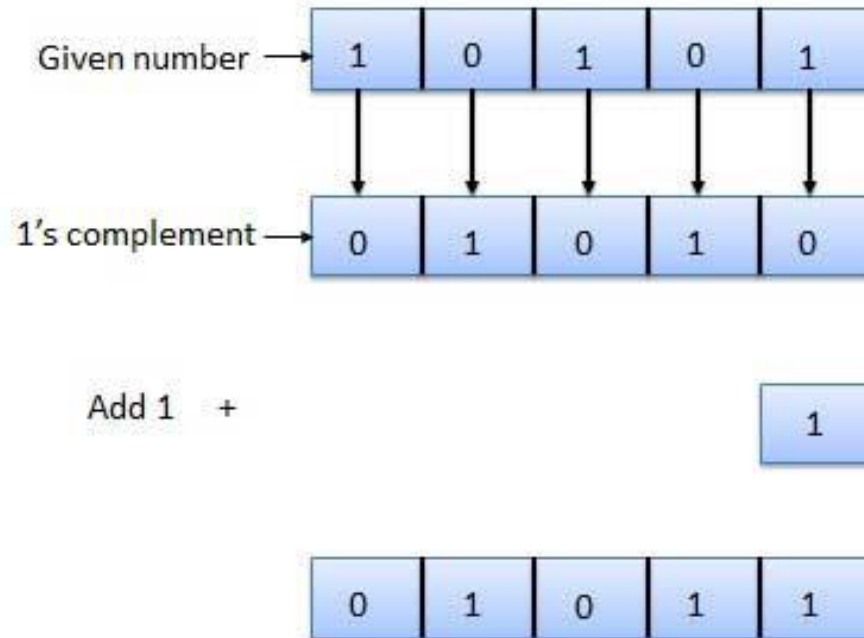
Binary system complements

- As the binary system has base $r = 2$. So the two types of complements for the binary system are 2's complement and 1's complement.
- **1's complement**
- The 1's complement of a number is found by changing all 1's to 0's and all 0's to 1's. This is called as taking complement or 1's complement.
- Example of 1's Complement is as follows.



Binary system complements

- **2's complement**
- The 2's complement of binary number is obtained by adding 1 to the Least Significant Bit (LSB) of 1's complement of the number.
- $2\text{'s complement} = 1\text{'s complement} + 1$
- Example of 2's Complement is as follows.



Binary subtraction using 1's complement

The 1's complement of a number is obtained by interchanging every 0 to 1 and every 1 to 0 in a binary number.

For example, the 1's complement of the binary number 110 is 001.

To perform binary subtraction using 1's complement, please follow the below steps

1. At first, find 1's complement of the B(subtrahend).
2. Then add it to the A(minuend).
3. If the final carry over of the sum is 1, then it is dropped and 1 is added to the result.
4. If there is no carry over, then 1's complement of the sum is the final result and it is negative.

Complements are used in digital circuits, because it is faster to subtract by adding complements than by performing true subtraction.

Binary subtraction using 1's complement

- Subtract 110010 - 100101
 - Step 1: Find out the 1's complement of the subtrahend (37), which is **011010**
 - Step 2: Add it with the minuend(50), which is 110010

$$\begin{array}{r} 011010 \\ + 110010 \\ \hline 1001100 \end{array}$$

- The left-most digit 1 is a carryover of this addition. Since there is a carryover we add it with the result, which is 00110

$$\begin{array}{r} 001100 \\ + 1 \\ \hline 001101 \end{array}$$

Binary subtraction using 1's complement

Find Subtraction of 110 and 101 using 1's complement method

- Step 1: Find out the 1's complement of the subtrahend which is **010**
- Step 2: Add it with the minuend which is 110

$$\begin{array}{r} 1 \\ 110 \\ + 010 \end{array}$$

$$1000$$

- The left-most digit 1 is a carryover of this addition. Since there is a carryover we add it with the result, which is 000

$$\begin{array}{r} 000 \\ + 1 \end{array}$$

$$001$$

Binary subtraction using 1's complement

Find Subtraction of 10110 and 11101 using 1's complement method

Step 1: Find out the 1's complement of the subtrahend which is **00010**

Step 2: Add it with the minuend which is 10110

$$\begin{array}{r} 11 \\ 10110 \\ + 00010 \\ \hline 11000 \end{array}$$

- Here there is no carry, answer is - (1's complement of the sum obtained 11000)
 - So answer is -00111
-

Binary subtraction using 1's complement

- **1011.001 – 110.10**

- **Solution:**

- 1's complement of 0110.100 is 1001.011 Hence

- Minued - 1 0 1 1 . 0 0 1

1's complement of subtrahend - 1 0 0 1 . 0 1 1

Carry over - 1 0 1 0 0 . 1 0 0

1

0 1 0 0 . 1 0 1

- **Hence the required difference is 100.101**
-

Binary subtraction using 1's complement

- **10110.01 – 11010.10**

- **Solution:**

- 1's complement of 11010.10 is 00101.01

- $$\begin{array}{r} 10110.01 \\ - 11010.10 \\ \hline \end{array}$$

$$\begin{array}{r} 10110.01 \\ - 11010.10 \\ \hline 00101.01 \end{array}$$

$$\begin{array}{r} 10110.01 \\ - 11010.10 \\ \hline 00101.01 \\ 11011.10 \end{array}$$

- **Hence the required difference is – 00100.01 i.e. – 100.01**
-

Binary subtraction using 2's complement

Method : 2's complement subtraction steps :

1. At first, find 2's complement of the B(subtrahend).
 2. Then add it to the A(minuend).
 3. If the final carry over of the sum is 1, then it is dropped and the result is positive.
 4. If there is no carry over, then 2's complement of the sum is the final result and it is negative.
-

Binary subtraction using 2's complement

Find Subtraction of 110 and 101 using 2's complement method

Step 1 : 2's complement of a number is 1 added to it's 1's complement number.

101 $\rightarrow$ 010 (1's complement)

010

+ 1

—

0 1 1 (2's complement)

Step 2: Now Add this 2's complement to 110

1

1 1 0

+ 0 1 1

—

1 0 0 1

Step 3: The left most bit of the result is called carry and it is ignored.

So answer is 001

Binary subtraction using 2's complement

Find Subtraction of 10110 and 11101 using 2's complement method

Step 1 : 2's complement of a number is 1 added to it's 1's complement number.

11101 $\rightarrow$ 00010 (1's complement)

00010

+ 1

00011 (2's complement)

Step 2: Now Add this 2's complement to 10110

11

10110

+ 00011

11001

Step 3: Here there is no carry, answer is - (2's complement of the sum obtained 11001)

00110 (1's complement)

00110+1 =00111 (2's complement)

So answer is -00111

Binary subtraction using 2's complement

110110 - 10110

Solution:

- The numbers of bits in the subtrahend is 5 while that of minuend is 6. We make the number of bits in the subtrahend equal to that of minuend by taking a '0' in the sixth place of the subtrahend.
- Now, 2's complement of 010110 is $(101001 + 1)$ i.e. 101010. Adding this with the minuend.

1 1 0 1 1 0 Minuend

1 0 1 0 1 0 2's complement of subtrahend

Carry over 1 1 0 0 0 0 0 Result of addition

- After dropping the carry over we get the result of subtraction to be 100000.
-

Binary subtraction using 2's complement

- **10110 – 11010**

- **Solution:**

- 2's complement of 11010 is $(00101 + 1)$ i.e. 00110. Hence

- Minued - 1 0 1 1 0

2's complement of subtrahend - 0 0 1 1 0

Result of addition - 1 1 1 0 0

- As there is no carry over, the result of subtraction is negative and is obtained by writing the 2's complement of 11100 i.e. $(00011 + 1)$ or 00100.
 - Hence the difference is – 100.
-

Binary subtraction using 2's complement

- **1010.11 – 1001.01**

- **Solution:**

- 2's complement of 1001.01 is 0110.11. Hence

- Minued - 1 0 1 0 . 1 1

2's complement of subtrahend - 0 1 1 0 . 1 1

Carry over 1 0 0 0 1 . 1 0

- After dropping the carry over we get the result of subtraction as 1.10.
-

Binary subtraction using 2's complement

- $10100.01 - 11011.10$

- **Solution:**

- 2's complement of 11011.10 is 00100.10. Hence

- $$\begin{array}{r} \text{Minued -} \\ 10100.01 \end{array}$$

$$\begin{array}{r} \text{2's complement of subtrahend -} \\ 00100.10 \end{array}$$

$$\begin{array}{r} \text{Result of addition -} \\ 11000.11 \end{array}$$

- As there is no carry over the result of subtraction is negative and is obtained by writing the 2's complement of 11000.11.
 - Hence the required result is -00111.01 .
-

Binary subtraction using 1's & 2's complement

- $(45)_{10} - (56)_{10} = (?)_2$ (with 2's complement)
 - $(56)_{10} - (45)_{10} = (?)_2$ (with 1's complement)
 - $(4545)_{10} - (5656)_{10} = (?)_2$ (with 2's complement)
 - $(5656)_{10} - (4545)_{10} = (?)_2$ (with 1's complement)
 - $(45.25)_{10} - (56.20)_{10} = (?)_2$ (with 2's complement)
 - $(35.21)_{10} - (21.35)_{10} = (?)_2$ (with 1's complement)
-

◆ What is Weighted Code?

A **Weighted Code** is a type of **binary code** in which **each bit (digit)** is assigned a **fixed positional weight**. These weights determine the **value** that each bit contributes to the final number.

👉 In simple words:

Each position in the binary number has a fixed value (weight), such as 8, 4, 2, 1, and the total number is found by adding the weights of all bits that are 1.

📊 Example: Binary Weighted Code (8-4-2-1)

Let's take the binary number:

`1011`

Each bit has a fixed weight from right to left:

Bit Position	2^3	2^2	2^1	2^0
Weight	8	4	2	1
Bit Value	1	0	1	1

Now multiply each bit by its weight:

$$\begin{aligned} &\rightarrow (1 \times 8) + (0 \times 4) + (1 \times 2) + (1 \times 1) \\ &= 8 + 0 + 2 + 1 \\ &= 11 \text{ (decimal)} \end{aligned}$$

✅ So, `10112 = 1110`

◆ Another Example: BCD (Binary Coded Decimal)

In BCD, each decimal digit (0–9) is represented by 4 bits using weights 8, 4, 2, 1.

Example: Decimal number 25

- 2 → 0010 (8-4-2-1)
- 5 → 0101 (8-4-2-1)

So,

$25_{10} = 0010\ 0101$ (BCD)

✓ Each bit has weights 8, 4, 2, 1 → hence it's a **Weighted Code**.

💡 In Short:

Weighted Code = A code where each position has a fixed weight (like 8,4,2,1) used to represent decimal numbers in binary form.

Example 1:

Convert 59_{10} to BCD

Step 1 Write each decimal digit separately

5 → 0101

9 → 1001

Step 2 Combine

✓ BCD = 0101 1001

Example 2:

Convert 204_{10} to BCD

Step 1 Separate digits: 2, 0, 4

2 → 0010

0 → 0000

4 → 0100

Step 2 Combine

✓ BCD = 0010 0000 0100

Example 3:

Convert 873_{10} to BCD

8 → 1000

7 → 0111

3 → 0011

✓ BCD = 1000 0111 0011

Example 4 (with decimal point):

45.7_{10}

Each digit is coded separately, the decimal point is not coded — it's just placed in the same position.

4 → 0100

5 → 0101

. → (decimal point)

7 → 0111

✓ BCD = 0100 0101 . 0111

Number System

To a computer, everything is a number, i.e., alphabets, pictures, sounds, etc., are numbers. Number system is categorized into four types –

- Binary number system consists of only two values, either 0 or 1
- Octal number system represents values in 8 digits.
- Decimal number system represents values in 10 digits.
- Hexadecimal number system represents values in 16 digits.

System	Base	Digits
Binary	2	0 1
Octal	8	0 1 2 3 4 5 6 7
Decimal	10	0 1 2 3 4 5 6 7 8 9
Hexadecimal	16	0 1 2 3 4 5 6 7 8 9 A B C D E F

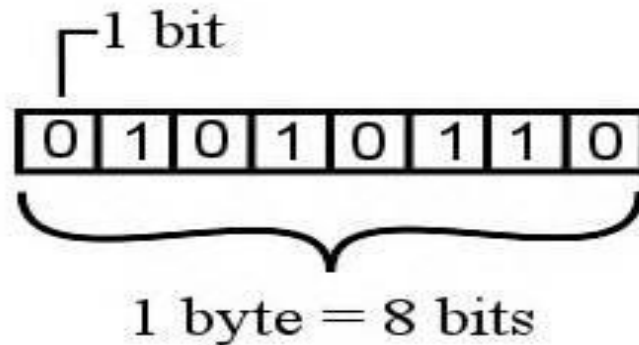
Number System

- The smallest unit of information that can be stored by a computer is a *binary digit* (usually shortened to "bit").
 - Its value is usually held in memory as an electrical charge stored in a capacitor. Modern memory chips contain millions of these tiny capacitors, each of which is capable of storing exactly one bit of information.
 - A single bit can have one of two values at any given time - *one* or *zero*.
 - As we shall see, in order to represent a number greater than one, we will have to use several bits.
-

Bits and Bytes

Bits – A bit is a smallest possible unit of data that a computer can recognize or use.
Computer usually uses bits in groups.

Bytes – group of eight bits is called a byte.
Half a byte is called a nibble.



Bits and Bytes

- Generally speaking, single bits are used only for storing *Boolean values* (true or false).
 - In most programming languages, *true* equates to *one*, while *false* equates to *zero*.
 - The definition of the byte has varied over the years but it is now generally considered to be a group of eight bits, and can be used to represent alpha-numeric and non-printing characters, unsigned integer (whole number) values from 0 to 255, or signed integer values from -127 to +127.
-

Bits and Bytes

- The number of bits that can be processed by a CPU in a single machine operation is dependent upon the number of bits it can store in its internal registers.
 - In the early days of computing this was a relatively small number (four or eight bits).
 - At some point, therefore, the size of the processor's register coincided with the size of a byte.
 - For many years now, however, this has not been the case.
 - As CPU architecture has advanced, we have seen the size of registers double and re-double. Most processors now have either 32-bit or 64-bit registers that can hold four or eight bytes of data respectively.
-

How Many Bits Are Necessary to Represent Something?

- 1 bit can represent two (2^1) symbols
 - either a 0 or a 1
- 2 bits can represent four (2^2) symbols
 - 00 or 01 or 10 or 11
- 3 bits can represent eight (2^3) symbols
 - 000 or 001 or 011 or 111 or 100 or 110 or 101 or 010
- 4 bits can represent sixteen (2^4) symbols
- 5 bits can represent 32 (2^5) symbols
- 6 bits can represent 64 (2^6) symbols
- 7 bits can represent 128 (2^7) symbols
- 8 bits (a byte) can represent 256 (2^8) symbols
- n bits can represent (2^n) symbols!

Bits and Bytes



A modern CPU has a 64-bit architecture

The following table shows conversion of Bits and Bytes –

Byte Value	Bit Value
1 Byte	8 Bits
1024 Bytes	1 Kilobyte
1024 Kilobytes	1 Megabyte
1024 Megabytes	1 Gigabyte
1024 Gigabytes	1 Terabyte
1024 Terabytes	1 Petabyte
1024 Petabytes	1 Exabyte
1024 Exabytes	1 Zettabyte
1024 Zettabytes	1 Yottabyte
1024 Yottabytes	1 Brontobyte
1024 Brontobytes	1 Geopbytes



Binary Coded Decimal (BCD)

- *Representing* one decimal number at a time
- How can we *represent* the ten decimal numbers (0-9) in binary code?

<u>Numeral</u>	<u>BCD Representation</u>
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Binary
Coded
Decimal

- We can represent any integer by a string of binary digits.
 - For example, 749 can be *represented* in binary as: 011101001001
-

Binary Coded Decimal (BCD)

- **Binary Coded Decimal**, or **BCD**, is another process for converting decimal numbers into their binary equivalents.
- It is noticeable that the BCD is nothing more than a binary representation of each digit of a decimal number.

